

Boring Financial 智能账单分类系统

项目成员： 郑博文、林子尧、单彬、沈柔希

项目周期： 2026 年 3 月 – 2026 年 6 月

目录

第一章 项目概述	1
1.1 项目背景与问题分析	1
1.2 项目目标	1
1.3 项目范围	3
1.3.1 范围内功能	3
1.3.2 范围外内容	4
1.4 项目收益	4
第二章 可行性分析与规划	5
2.1 开发流程与选择依据	5
2.1.1 迭代阶段与反馈闭环	5
2.2 可行性分析	7
2.2.1 方案选型分析	8
2.3 项目计划与进度	8
2.3.1 工作分解结构 (WBS)	8
2.3.2 里程碑计划	9
2.3.3 进度甘特图	10
2.4 团队组织与协作	10
2.4.1 团队成员与分工	10
2.4.2 人员变动预案	11
2.4.3 协作机制	11
2.5 风险管理	12
2.5.1 变更管理策略	13
第三章 需求分析	14
3.1 需求获取方法	14
3.2 利益相关者分析	15
3.3 功能需求	15
3.4 非功能需求	17
3.5 用户场景分析	18
3.5.1 普通用户场景	18
3.5.2 家庭组织场景	18

3.5.3	系统运维场景	19
3.6	异常场景处理	19
3.7	用例建模	20
3.8	用例规约	21
3.8.1	UC-01: 导入账单与语义分类	21
3.8.2	UC-02: 人工校正分类结果	22
3.9	业务流程建模	22
第四章	用户界面与体验设计	24
4.1	设计迭代过程	24
4.2	设计系统	24
4.3	主要页面原型	25
4.3.1	登录与注册页面	25
4.3.2	仪表盘页面	26
4.3.3	账单导入页面	27
4.3.4	交易列表页面	27
4.3.5	分类校正工作台	28
4.3.6	分类管理页面	28
4.3.7	消费人格页面	29
4.3.8	家庭组织页面	29
4.3.9	报表中心页面	30
4.3.10	系统设置页面	30
4.4	界面信息架构与交互组织	31
4.5	用户体验目标	31
第五章	系统设计	33
5.1	总体架构	33
5.2	技术选型	34
5.3	前端架构	34
5.4	后端架构	36
5.5	数据隔离策略	36
5.6	部署架构	36
5.7	安全机制	38
第六章	程序开发	39
6.1	面向对象程序组织	39
6.1.1	领域实体及其关联关系	39

6.1.2	分类器策略	39
6.1.3	账单解析器	41
6.1.4	报表构建器	41
6.2	数据库设计	41
6.2.1	核心账单与分类	42
6.2.2	家庭组织与报表扩展	42
6.3	设计模式应用	44
6.4	核心流程设计	45
6.4.1	账单导入与大模型初分	45
6.4.2	家庭组织聚合查询	45
第七章	验收与发布	47
7.1	自动化测试	47
7.1.1	静态代码验证	47
7.1.2	动态测试	47
7.2	测试结果	48
7.3	验收测试	49
7.4	发布状态	50
第八章	运行与维护	52
8.1	运行环境	52
8.2	性能管理	52
8.3	可靠性保障	53
8.4	运维检查	54
第九章	总结与改进计划	55
9.1	项目完成情况	55
9.2	当前局限与后续改进	55

第一章 项目概述

1.1 项目背景与问题分析

在个人财务管理场景中，自动记账工具（如微信账单导出、支付宝账单导出）已成为日常财务整理的重要手段。然而，传统记账软件在自动分类环节存在显著短板，直接影响统计分析的准确性。一个典型且普遍的场景是：当用户向校园一卡通充值时，账单上的商户名称可能仅显示为「东南大学」。基于关键词匹配的传统分类引擎很容易将其归入「学习」类别，但从真实消费意图来看，这笔交易更接近于「餐饮/校园卡充值」。这种语义误判一旦发生，不仅会扭曲用户的消费结构认知，还会在没有校正机制的情况下持续污染后续的财务统计。

进一步分析，个人用户在整理微信、支付宝账单时通常会遇到三类共性问题：

1. **自动分类依赖浅层规则**：传统分类引擎仅根据商户名或关键词做简单匹配，难以理解「在便利店买口罩」和「在便利店买零食」之间的消费意图差异。
2. **分类依据不透明**：用户不知道一笔交易为什么被归入某个类别，缺乏解释性，因而无法判断分类是否可信。
3. **误判缺少校正入口**：当分类出错时，用户要么被动接受错误结果，要么需要逐个手动修改，缺乏一个高效、可控的人工校正机制。

Boring Financial 正是针对上述问题设计的解决方案。系统围绕「导入—大模型语义初分—人工校正—多维度分析—报表输出」的完整闭环展开，将原始账单转换为语义准确、可解释、可修正的财务分析结果，同时引入消费人格画像和家庭组织聚合等创新分析维度，为用户提供比传统记账工具更深层的财务洞察。

1.2 项目目标

Boring Financial 的核心定位是一个面向个人与家庭的智能账单分类与财务分析系统。项目的具体目标如表 1.1 所示。

表 1.1 项目目标

目标	说明
完成记账分析闭环	支持账单导入、格式解析、大模型语义初分、人工校正、统计看板、消费人格画像、家庭组织聚合和 PDF 报表的完整流程。
支持多用户与数据隔离	用户默认只能访问自己的账单、交易、分类和报表；加入家庭组织后可按成员角色查看聚合数据，个人数据的独立性不受影响。
支持语义分类与人工校正	以大模型语义分类为主要手段，同时保留规则分类作为兜底方案，并为低置信度交易提供人工校正入口，确保用户对最终分类结果拥有控制权。
支持工程化部署	提供清晰的系统架构、接口文档、开发指南、测试套件和部署说明，支持从本地开发到 Docker Compose 裸机部署的多种运行模式。
控制实现复杂度	聚焦记账分析闭环本身，不引入支付、清算和银行级风控等超出项目范围的功能。

1.3 项目范围

1.3.1 范围内功能

表 1.2 项目范围内功能模块

模块	范围说明
用户认证	注册、登录、JWT 鉴权、多用户数据隔离。
账单导入	支持微信、支付宝账单文件上传，管理导入批次状态和导入记录。
账单解析	将不同平台的原始账单格式转换为统一的交易数据结构，支持去重。
大模型初分	支持规则分类、缓存分类和外部大模型语义分类的组合链路，提供分类理由与置信度，含重试队列机制。
人工校正	用户可筛选和复核低置信度交易，修正错误分类，修正结果在后续统计中体现。
分类管理	支持系统预设分类和用户自定义分类，系统分类不可禁用。
财务看板	展示收入、支出、结余、储蓄率、分类占比分布、Top 商户和收支趋势。
消费人格	基于行为经济学理论生成 16 型消费人格画像、多维财务健康评分，并提供自评问卷与数据画像的偏差对比。
家庭组织	支持创建家庭组织、管理成员角色 (owner/admin/member)，聚合家庭 Dashboard、家庭消费人格和家庭报表。
PDF 报表	按日期范围和导入文件生成可下载的 PDF 财务报告。
系统部署	支持本地开发运行、Docker Compose 编排部署和裸机 Nginx + systemd 部署。

1.3.2 范围外内容

表 1.3 项目范围外内容及原因

不做内容	原因
支付、转账、清算	涉及真实资金流和金融合规要求，超出项目范围。
银行级风控与审计	需要严格的合规流程和高安全保障，不属于记账分析闭环的核心关注点。
多机构财务管理	当前定位为个人与家庭的账单分类与分析工具。
复杂家庭预算审批流程	当前家庭组织模块聚焦成员聚合和角色管理，不做多级审批流。
商业化运营后台	系统聚焦个人使用和轻量部署，不覆盖商业运营能力。

1.4 项目收益

Boring Financial 的收益可以从多个利益相关方视角进行审视，如表 1.4 所示。

表 1.4 项目收益分析

受益方	收益
普通用户	减少误分类带来的统计偏差，快速理解个人支出结构、消费趋势和消费行为特征。通过消费人格画像获得对自身消费心理的深层认知，通过家庭组织模块了解家庭成员的整体财务状况。
家庭管理员	可将个人账本聚合为家庭视图，了解家庭整体的收入支出情况和消费结构，无需为家庭成员创建共享账本或手动合并数据。
系统运维人员	基于清晰的模块边界、标准化的接口文档、自动化测试套件和多种部署方案，可快速搭建和维护系统运行环境。
项目开发团队	在实践中覆盖了需求分析、系统设计、编码实现、测试验证、部署运维和文档整理的全流程，积累了从单体应用到前后端分离系统的完整开发经验。

第二章 可行性分析与规划

2.1 开发流程与选择依据

本项目采用轻量级迭代开发流程，而非传统的瀑布模型。这一选择基于以下实际约束和工程考量：

表 2.1 开发流程选择依据

因素	说明
有限的项目周期	项目从 3 月持续至 6 月，需要在约四个月内完成从需求分析到部署交付的全部工作，要求流程具备快速反馈和持续交付能力。
需求随实现迭代演化	不同平台的账单格式差异、大模型分类的实际效果以及前端交互体验都需要在实际实现和联调过程中不断验证和修正，难以在初始阶段完全冻结需求。
团队可并行推进	前端、后端、分类模型、测试和文档等工作可按照模块边界并行开发，迭代流程天然支持这种并行协作模式。
MVP 优先策略	首先保证「导入—大模型初分—人工校正—分析—报表」的核心闭环可运行，后续再逐步补充增强功能。
文档持续积累	各阶段的过程文档在开发过程中持续积累，在最终交付阶段统一整理为完整报告，避免期末集中赶工。

2.1.1 迭代阶段与反馈闭环

系统的开发过程划分为四个阶段，阶段之间通过反馈回路进行修正：

- **阶段一（3 月）：选题与需求。**确立项目目标、完成可行性研究、明确团队分工和需求范围。核心产出包括项目目标说明、可行性论证、需求列表和利益相关者分析。
- **阶段二（4 月）：原型与架构。**完成前端页面原型、系统架构设计、组件图、部署图，以及安全性和性能的初步设计。此阶段的架构决策为后续实现提供了稳定的技术骨架。
- **阶段三（5 月）：功能实现与联调。**完成核心功能的开发并进行前后端联调。实现范围覆盖用户认证、账单导入、AI 分类、人工校正、Dashboard、消费人格画像、

家庭组织和 PDF 报表。实现过程中发现的技术约束会反向修正架构设计，联调中发现的问题会反向修正需求定义。

- **阶段四（6 月）：测试、部署与报告。**完成自动化测试和手工验收测试，编写部署文档，整理最终报告和答辩材料。测试中发现的缺陷会触发对实现和设计的修正。

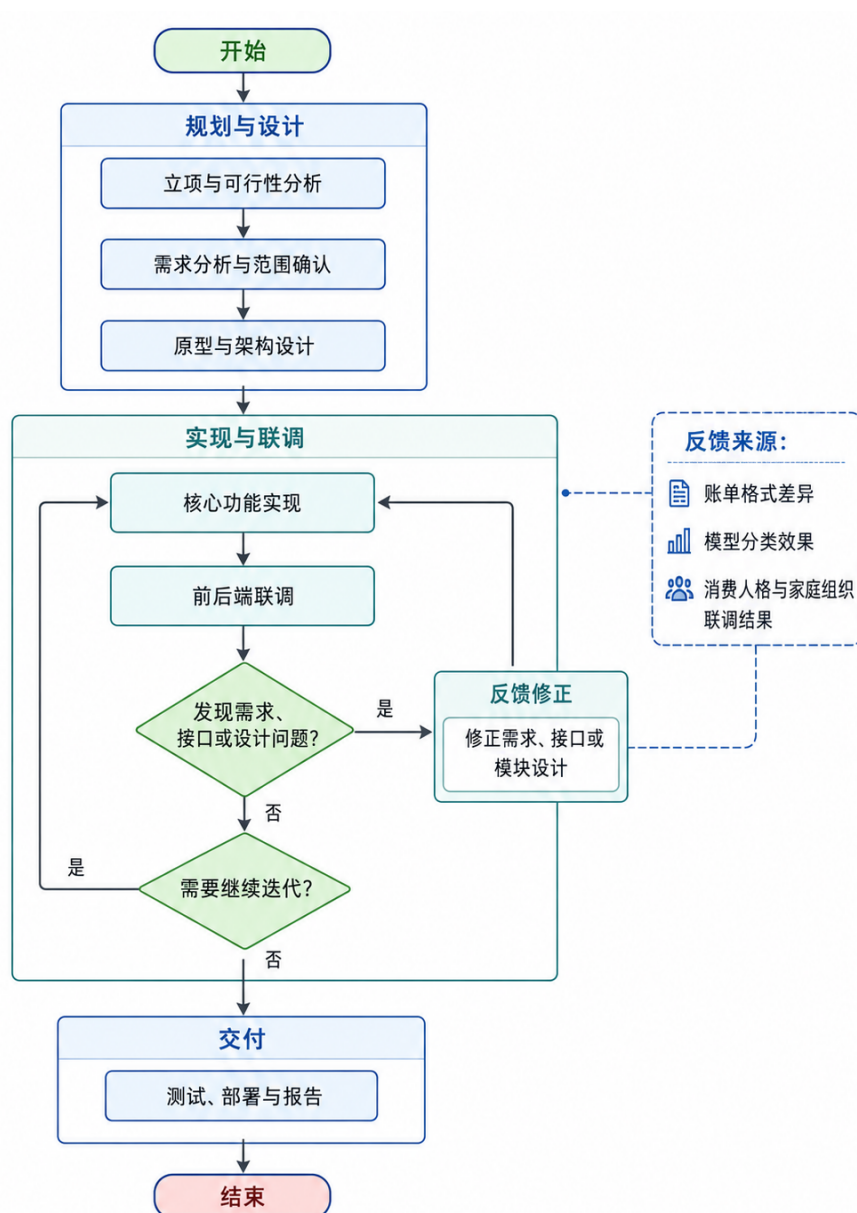


图 2.1 开发流程迭代图

2.2 可行性分析

在项目启动阶段，从五个维度对项目可行性进行了系统评估。评估结果如表 2.2 所示。

表 2.2 多维度可行性分析

维度	结论	依据
业务可行性	可行	自动记账分类误判是真实的用户痛点，大模型语义分类结合人工校正的解决方案价值明确。消费人格画像和家庭组织聚合进一步增强了产品的差异化竞争力。
技术可行性	可行	技术栈选择成熟稳定：FastAPI 提供高性能异步 API，Vue 3 + Element Plus 提供组件化的前端能力，SQLAlchemy 提供 ORM 层的数据操作抽象，Docker 提供一致化的运行环境。所有技术均有丰富的社区支持和生产实践验证。
进度可行性	可行	四个月的项目周期可以支撑四阶段的迭代开发节奏。四人团队按前端、后端、模型/测试的模块边界并行推进，核心闭环可以在两个月内完成 MVP。
资源可行性	可行	开发设备使用团队成员的本地电脑，无需额外硬件投入。Git、Docker、PostgreSQL、Redis 等工具均采用开源或免费方案。模型服务支持本地 mock 和规则兜底，可灵活控制对商用 API 的依赖。
经济可行性	可行	项目预算极低，主要成本为成员的开发时间和可选的大模型 API 调用费用（可通过规则兜底和本地模型服务进一步压缩）。
维护可行性	基本可行	采用 monorepo 代码组织、分层后端架构和前端组件化设计，配合接口文档和自动化测试，为后续维护提供了可追溯的技术基础。

2.2.1 方案选型分析

在系统核心分类策略上，项目初期评估了三种备选方案，如表 2.3 所示。

表 2.3 分类策略方案对比

方案	优点	缺点	结论
仅关键词规则分类	稳定可控，实现简单	难以理解「东南大学饭卡充值」等语义场景，误判后统计失真	不作主要方案
仅大模型自动分类	语义理解能力强	用户缺少最终控制权，模型错误直接污染统计	不作完整方案
大模型初分 + 人工校正	兼顾语义理解、可解释性和用户控制权	实现复杂度更高	最终方案

最终选择「大模型语义初分 + 人工校正」的前后端分离架构，同时保留规则分类作为兜底方案和重试队列作为容错机制。该方案在自动分类的语义准确性与用户对关键错误的最终控制权之间取得了平衡。

2.3 项目计划与进度

2.3.1 工作分解结构（WBS）

项目的核心任务分解如表 2.4 所示。各任务按模块边界划分，明确负责人、前置依赖和交付物。

表 2.4 项目工作分解结构（WBS）

编号	任务	负责人	前置依赖	交付物
WBS-01	项目目标与范围确认	郑博文	无	项目立项与可行性资料
WBS-02	需求分析与规格说明	郑博文、全体	WBS-01	需求规格文档
WBS-03	后端数据模型与接口设计	林子尧	WBS-02	ORM 模型、API 路由定义

接下页

表 2.4 项目工作分解结构（续）

编号	任务	负责人	前置依赖	交付物
WBS-04	账单解析与分类服务	林子尧、沈柔希	WBS-03	Parser、Classifier 实现
WBS-05	前端页面与交互实现	单彬	WBS-02、WBS-03	全部业务前端页面
WBS-05A	家庭组织协作能力	林子尧、单彬	WBS-03、WBS-05	Organization API 与页面
WBS-06	模型 provider 与重试机制	沈柔希	WBS-04	Provider 配置、重试队列
WBS-07	前后端联调	全体	WBS-03、WBS-05	可运行系统 demo
WBS-08	测试与质量保证	沈柔希、全体	WBS-07	pytest 报告、验收记录
WBS-09	部署与运行文档	沈柔希、林子尧	WBS-07	部署说明文档
WBS-10	最终报告与答辩材料	郑博文、全体	WBS-01–WBS-09	最终报告、答辩 PPT

2.3.2 里程碑计划

项目设置了五个关键里程碑节点，对齐各阶段的验收要求，如表 2.5 所示。

表 2.5 项目里程碑计划

里程碑	时间	阶段	成功标准
M1	4 月上旬	阶段一	项目名称、团队、可行性、进度和需求分析可展示
M2	4 月下旬	阶段二	用户体验设计、系统架构、安全和性能设计可说明
M3	5 月中旬	阶段三	IDE 配置、组件复用、类图、顺序图和设计模式可解释
M4	5 月下旬	阶段三/四	测试覆盖率、单元/集成/验收测试、可靠性设计可验证
M5	6 月	阶段四	系统 demo、测试结果、部署资料和最终报告完整交付

2.3.3 进度甘特图

图 2.2 展示了项目从 3 月到 6 月的整体进度安排和各阶段的并行关系。

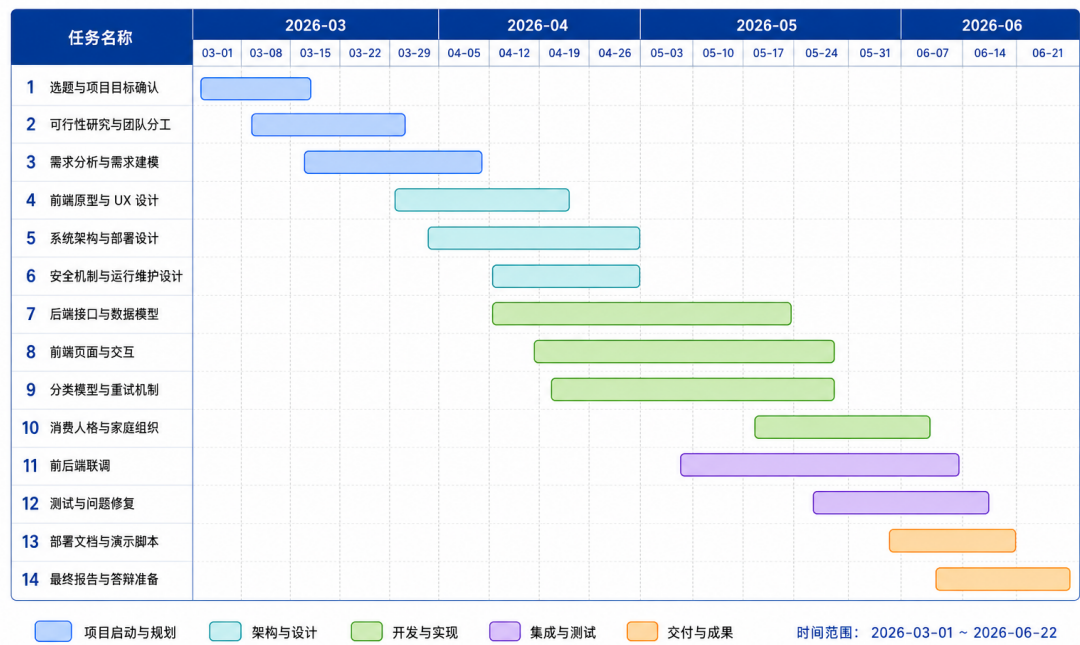


图 2.2 项目进度甘特图

2.4 团队组织与协作

2.4.1 团队成员与分工

项目由四名成员协作完成，各成员依据技术专长承担主要职责，并通过文档和规范降低人员交接成本。成员分工如表 2.6 所示。

表 2.6 团队成员与分工

成员	主要职责	备份与协作机制
郑博文	项目管理、需求统筹、答辩材料整理	维护进度计划和课程要求对照表，协调文档合并与版本管理。
林子尧	后端开发、数据库设计、接口实现	通过接口文档和自动化测试用例降低后端代码的交接成本。
单彬	前端页面、交互流程、图表展示	通过组件拆分和统一的 API 客户端封装降低前端页面的维护成本。
沈柔希	分类模型、测试、部署与联调	维护分类策略配置、测试流程和部署脚本，确保模型链路和部署方案可复现。

2.4.2 人员变动预案

为应对开发过程中可能出现的人员变动风险，团队制定了表 2.7 所示的应急预案。

表 2.7 人员变动应急预案

风险场景	应对方式
某成员生病或临时缺席	通过 Git 提交记录、接口文档和任务拆分让其他成员接手其负责模块的小范围修复工作。
某模块开发严重滞后	优先保证 MVP 核心闭环可演示，暂缓模块内的增强功能，待核心流程稳定后再补充。
答辩前关键资料缺失	以 README、项目文档、汇报 PPT 和测试结果作为可追溯的证据链，确保交付物完整性。

2.4.3 协作机制

团队协作依托以下工程实践：

- **版本控制**：使用 Git 进行代码版本管理，所有代码变更通过 Pull Request 合并，保持主分支可运行。
- **接口约定**：前后端以 OpenAPI 文档为契约，后端接口变更需同步更新文档，前端基于接口文档独立开发。

- **每日同步**：通过即时通讯工具保持进度同步，关键设计决策通过集中讨论确定并记录在项目文档中。
- **文档持续化**：需求、设计、测试和部署等各阶段文档随开发过程持续撰写和更新，避免期末集中整理。

2.5 风险管理

项目启动阶段对主要风险进行了识别和评估，并制定了相应的缓解措施。风险矩阵如表 2.8 所示。

表 2.8 项目风险矩阵

风险	概率	影响	等级	缓解措施
微信、支付宝账单格式差异	高	中	高	使用 ParserRegistry 模式处理多平台格式，所有解析器输出统一交易结构，保留测试样例。
大模型分类结果不稳定	中	中	中	在交易详情中展示分类理由和置信度；保留规则分类兜底和分类缓存；低置信度交易进入人工校正队列。
外部模型 API 不可用或延迟高	中	中	中	支持本地模型服务；外部请求通过统一重试队列串行处理，超时耗尽后标记为 retry_failed；可随时切回规则分类。
家庭组织权限配置错误	中	高	高	使用 OrganizationMember 角色校验 (owner/admin/member)；个人账本默认不受组织功能影响；所有组织接口校验成员身份。
后期时间不足	中	高	高	坚持 MVP 优先策略，先保证核心闭环可运行，再补充增强功能和文档润色。
多人协作接口不一致	中	中	中	以 OpenAPI 文档为前后端契约；后端测试覆盖核心接口；使用类型定义和统一 API 客户端约束前端请求。
安全设计不足	中	高	高	使用 JWT 鉴权、passlib 密码哈希、多用户 user_id 隔离；后续安全章节中列出已知改进项和加固建议。
大文件导入性能不足	中	中	中	当前限制上传文件规模；后续可通过异步任务、批量写入和 SQL 聚合优化大批量数据处理性能。

2.5.1 变更管理策略

在迭代开发过程中，各类变更按照表 2.9 所示的策略进行处理，以确保变更不会冲击核心闭环的稳定性。

表 2.9 变更管理策略

变更类型	处理方式
新增功能需求	先判断是否影响 MVP 核心闭环；若不影响则放入后续增强列表，优先保证当前阶段交付。
接口字段变化	后端更新接口文档（自动生成的 OpenAPI schema），前端同步修改 API 客户端和类型定义。
家庭组织权限变化	先保证个人账本默认行为不变，在此基础上扩展 organization_id 聚合查询的权限校验逻辑。
分类策略变化	保留规则兜底，记录每次分类的 provider、置信度和分类理由，确保分类链路可追溯。
页面交互变化	优先不破坏主流程（导入 → 分类 → 分析 → 报表），避免在交付前进行大范围 UI 改版。
部署环境变化	同时维护本地直接启动和 Docker Compose 两套方案，保障在任何环境下系统均可演示。

第三章 需求分析

3.1 需求获取方法

本系统的需求通过五种方法获取，覆盖从用户行为分析到工程质量审查的完整链路。

真实账单处理流程分析：项目启动阶段，团队收集支付宝与微信支付的原始账单文件，模拟用户从下载账单、整理数据到生成报表的完整操作流程，提炼出导入、解析、分类、分析、报表五大核心环节，作为功能需求的主要依据。

小组讨论与范围界定：团队依据开发周期与技术能力，对需求进行优先级排序，将核心闭环（认证、导入、分类、分析、报表）确定为最高优先级，将消费人格分析、家庭组织、模型切换等纳入扩展范围，形成清晰的 MVP 边界。

误分类案例分析：团队收集了多种语义误判场景——如“东南大学饭卡充值”被通用分类归为“学习”、连锁便利店消费被归为“购物”而非“餐饮”——驱动了大模型初分、分类理由展示和人工校正工作台的需求。系统需在给出分类建议的同时，提供置信度与推理理由，允许用户在校正工作台中批量修正。

原型与联调反馈：在前后端迭代过程中，前端页面的交互细节和后端接口的返回结构经过多轮联调调整。例如，导入页面的进度轮询机制、校正工作台的左右分栏布局、家庭组织的角色权限控制，均来自原型验证阶段的反馈。

工程质量审查：团队对照安全、隐私、性能、可部署性、可维护性、可测试性等质量维度，识别并补齐非功能需求。这一步骤确保系统不仅是功能可用的原型，也是具备工程质量的交付物。

3.2 利益相关者分析

系统涉及四类利益相关者，其关注点与需求来源如表所示。

表 3.1 利益相关者分析

利益相关者	关注点	需求来源
普通用户	上传账单、查看 AI 分类结果、修正语义误判、查看消费画像、生成报表	个人记账与财务分析场景
家庭组织管理员	创建家庭组织、添加成员、管理角色、查看家庭聚合分析	家庭账本协作场景
系统运维人员	配置模型供应商、管理重试队列、部署与监控系统	系统部署与运维场景
开发与维护人员	实现系统功能、控制开发范围、维护代码质量、准备后续扩展	工程开发与维护过程

3.3 功能需求

系统共识别 15 项功能需求，按优先级分为 P0（核心闭环必备）、P1（完整性与体验增强）和 P2（未来改进）。当前实现覆盖全部 P0 与 P1 需求。

表 3.2 功能需求列表

ID	需求描述	优先级	验收依据
FR-01	用户注册、登录及身份验证	P0	用户可完成注册和登录，未登录状态下无法访问业务页面
FR-02	多用户数据隔离，账单与报表按用户归属	P0	不同账号只能查看和操作自己的交易、分类与报表
FR-03	支持微信/支付宝账单文件上传导入	P0	用户可上传账单文件，并看到导入进度和处理结果
FR-04	不同平台账单统一解析为标准化交易结构	P0	微信和支付宝账单均能转换为统一的交易列表
FR-05	大模型语义分类，记录来源、置信度与理由	P0	每笔交易显示建议分类、可信程度和可理解的分​​类理由

接下页

表 3.2 功能需求列表（续）

ID	需求描述	优先级	验收依据
FR-06	交易列表筛选（日期、平台、分类、关键词）与分页	P0	用户可按多条件检索交易，分页结果与筛选条件一致
FR-07	人工校正低置信度或语义误判交易	P0	用户可修改错误分类，校正后结果反映到分析与报表中
FR-08	用户自定义分类管理	P1	用户可新增、编辑和停用个人分类，系统分类保持可用
FR-09	Dashboard 展示收入、支出、结余、储蓄率、分类占比	P0	看板指标与导入后的交易数据一致，并支持时间范围切换
FR-10	消费人格画像、财务健康评分与问卷自评	P1	页面展示人格类型、维度评分，并支持自评结果对比
FR-11	家庭组织创建与成员角色管理	P1	管理员可创建家庭、添加成员并调整成员权限
FR-12	家庭组织 Dashboard 与消费人格聚合	P1	家庭视图可展示成员聚合后的收支概览与人格画像
FR-13	PDF 报表生成与下载，支持个人与家庭模式	P0	用户可按日期和账单范围生成、预览并下载报表
FR-14	分类供应商切换（规则/外部 API/本地模型）	P1	管理员可配置分类方式，模型不可用时系统仍能继续工作
FR-15	外部模型失败重试队列与管理员控制	P1	分类失败可自动重试，管理员可查看失败数量并触发恢复

FR-01 至 FR-07、FR-09、FR-13 构成系统的核心闭环：认证后导入账单并自动分类，经人工校对后产出 Dashboard 分析与 PDF 报表。FR-08、FR-10 至 FR-12、FR-14、FR-15 在此闭环上增加了消费人格画像、家庭协作和运维可控性。

3.4 非功能需求

系统定义了 8 项非功能需求，涵盖安全性、隐私保护、可用性、性能、可部署性、可维护性、可测试性与可扩展性。

表 3.3 非功能需求列表

ID	需求名称	优先级	说明
NFR-01	安全性	P0	密码采用哈希存储，业务接口均需 JWT 鉴权，资源访问按用户归属校验
NFR-02	隐私保护	P0	账单数据、分类结果与 PDF 报表属于敏感个人财务数据，用户间严格隔离
NFR-03	可用性	P0	用户可按“导入 → 初分 → 校正 → 分析 → 报表”路径完成核心任务；家庭功能不破坏个人模式
NFR-04	性能	P1	常规查询和聚合分析在单用户正常操作下应保持秒级响应；大模型分类设有超时和自动重试保护；千笔以内账单应能在可接受时间内完成解析并展示进度
NFR-05	可部署性	P0	支持本地开发模式、Docker Compose 部署与裸机部署三种方式
NFR-06	可维护性	P0	前后端职责分离，业务逻辑、数据模型、接口契约和部署配置边界清晰
NFR-07	可测试性	P0	核心功能具备自动化测试和构建验证，关键用户流程可通过验收清单复核
NFR-08	可扩展性	P1	分类供应商、报表生成、异步任务与部署组件预留扩展接口

3.5 用户场景分析

3.5.1 普通用户场景

普通用户的核心场景覆盖“导入—分类—校正—分析—报表”的完整链路：

1. 用户注册账号并登录系统，进入应用主界面。
2. 进入账单导入页面，上传微信或支付宝账单文件（CSV 格式）。
3. 系统创建导入批次，后台解析账单并将不同平台格式转换为统一交易记录。
4. 系统调用大模型对每条交易进行语义初分，记录分类来源、置信度与分类理由。
5. 用户在交易列表中按日期、平台、分类、关键词筛选查看交易。
6. 在分类校正工作台，用户查看待校正交易队列，对低置信度或语义误判的分类进行人工修正。
7. 用户进入 Dashboard 查看收入支出概览、储蓄率、分类占比图与 Top 商户。
8. 用户进入消费人格页面，查看基于交易数据计算的消费人格画像、财务健康评分，并可通过问卷进行自评对比。
9. 用户进入报表中心，选择日期范围与导入文件，生成并下载 PDF 报表。

3.5.2 家庭组织场景

家庭组织场景描述管理员创建家庭、管理成员并查看聚合分析的过程：

1. 用户创建家庭组织，系统自动将创建者设为 owner 角色。
2. owner 或 admin 通过用户名搜索添加成员，并可提升普通成员为 admin。
3. 家庭成员进入组织页面查看家庭合计收入、支出、交易数与消费人格聚合雷达图。
4. owner 或 admin 可触发家庭范围内的重分类操作；普通 member 只能查看聚合结果。
5. 用户未选择家庭视图时，看板、人格分析和报表仍按个人账本模式工作，保持个人使用路径不变。

3.5.3 系统运维场景

运维人员关注系统部署、配置与故障恢复：

- 1. 运维人员配置后端运行环境，包括数据库连接、Redis 缓存、模型供应商与 API 密钥。
- 2. 部署前端静态资源、Nginx 反向代理、FastAPI 后端、PostgreSQL 与 Redis。
- 3. 当模型请求超时或失败时，运维人员通过重试状态接口查看队列中的交易数量与供应商分布。
- 4. 运维人员通过管理界面将历史超时或失败交易重新放入重试队列，恢复分类流程。

3.6 异常场景处理

系统对以下异常场景定义了明确的预期行为：

表 3.4 异常场景与系统预期行为

异常场景	系统预期行为
用户未登录访问业务接口	返回 401 未授权状态码，前端跳转至登录页
用户上传不支持的账单格式	导入失败并记录具体错误信息，页面提示用户重新上传
重复导入同一笔交易	系统自动识别重复记录，避免重复统计和重复展示
模型供应商不可用	交易进入待重试状态，系统自动恢复；多次失败后提示管理员处理
用户尝试访问他人报表或交易	系统校验资源归属，拒绝非授权访问
报表生成失败	记录任务状态与错误信息，用户可调整参数后重新触发生成

3.7 用例建模

系统用例图展示三类参与者（普通用户、家庭管理员、系统运维人员）与系统功能之间的关系。普通用户可操作认证、导入、查看交易、校正分类、管理自定义分类、查看 Dashboard、生成报表、查看消费人格等用例；家庭管理员额外可创建组织和管理成员；系统运维人员负责供应商配置与重试队列管理。导入账单用例包含大模型语义分类作为扩展关系。

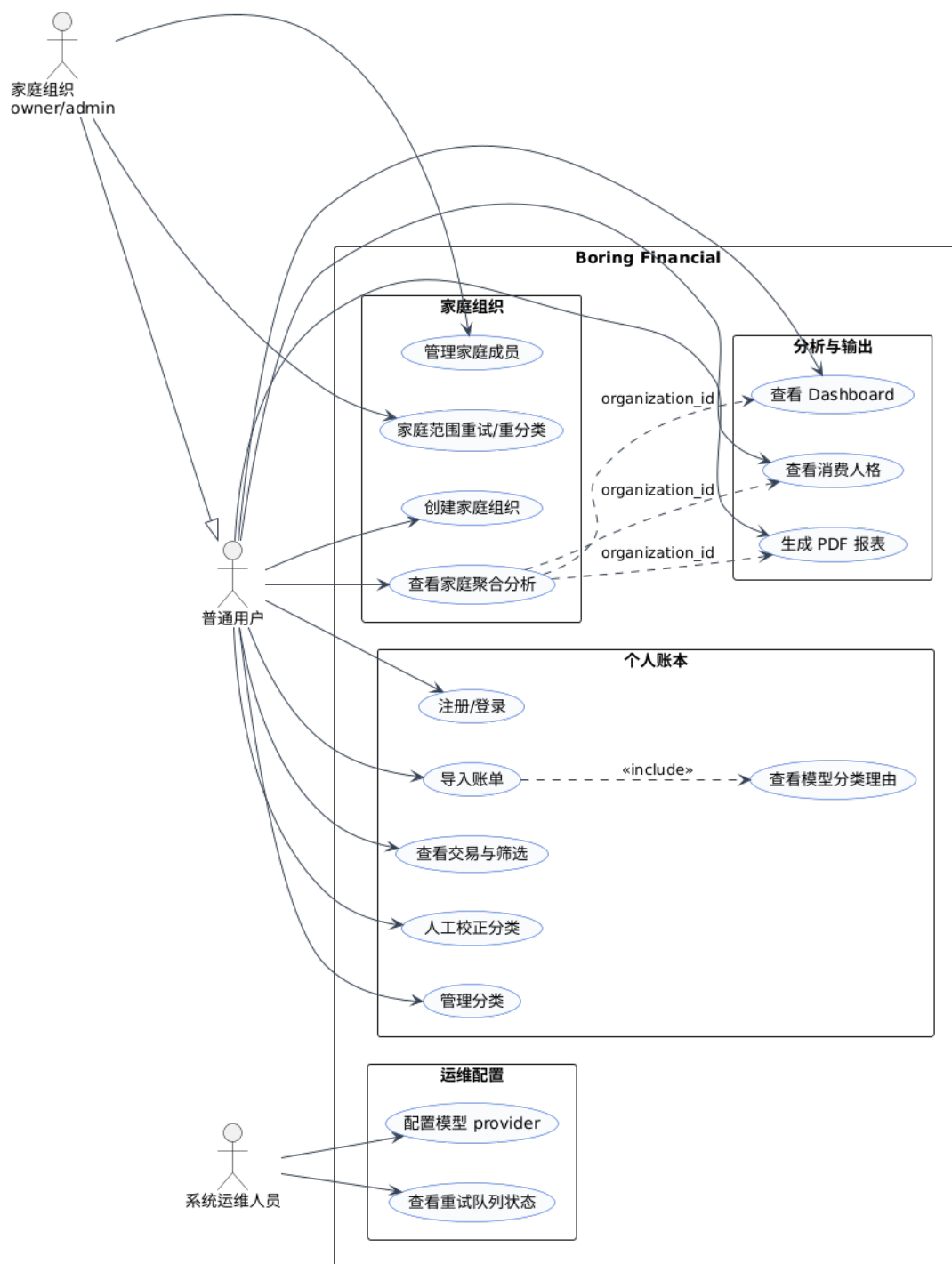


图 3.1 系统用例图

3.8 用例规约

以下对两个核心用例给出结构化用例说明，包含基本流程、替代流程与异常流程。

3.8.1 UC-01：导入账单与语义分类

表 3.5 用例规约：导入账单与语义分类

用例名称	导入账单与语义分类
参与者	普通用户（主）；大模型分类服务（外部系统）
触发条件	用户进入账单导入页面，选择账单文件并点击上传
前置条件	用户已登录；待上传文件为微信或支付宝 CSV 格式
后置条件	交易记录导入完毕，语义初分完成，页面展示成功或部分成功状态
基本流程	1) 用户拖拽或选择账单文件。 2) 系统接收文件并创建导入任务，页面开始展示处理进度。 3) 系统识别账单来源，将原始记录转换为统一交易格式。 4) 系统跳过已存在的重复交易，避免重复统计。 5) 系统对新交易进行语义分类，生成分类建议、置信度和理由。 6) 低置信度或失败交易进入待处理状态，正常交易进入后续统计。 7) 导入任务结束后，页面展示成功数量、失败数量和下一步操作入口。
替代流程 A	部分交易分类超时：系统保留已成功处理的交易，并将失败交易加入后续重试或人工处理范围。
替代流程 B	文件含已导入交易：系统自动跳过重复记录，批次仍正常完成。
异常流程	文件格式无法识别：系统停止本次导入，页面说明失败原因并提示用户重新上传。

3.8.2 UC-02：人工校正分类结果

表 3.6 用例规约：人工校正分类结果

用例名称	人工校正分类结果
参与者	普通用户
触发条件	用户进入分类校正工作台
前置条件	系统中存在低置信度或用户认为需要复核的交易记录
后置条件	目标交易的最终分类已更新，校正结果实时反映在看板、人格分析及报表数据中
基本流程	1) 页面加载待校正队列，并优先展示置信度较低的交易。 2) 用户选中一笔交易，右侧面板展示商户、金额、当前分类、置信度与模型推理理由。 3) 用户从系统分类或自定义分类中选择正确类别，点击”确认校正”。 4) 系统保存新的最终分类，并同步更新统计口径。 5) 页面从待校正队列移除该交易，自动加载下一笔。
替代流程	用户认为当前分类正确：直接点击”确认当前分类”，系统保留建议分类并将其视为最终结果。
异常流程	所需自定义分类尚不存在：用户跳转至分类管理页面新增分类后，返回工作台重新操作。

3.9 业务流程建模

核心业务活动图描述了从用户登录到报表生成的端到端流程。流程始于用户登录，接着上传账单文件，系统创建导入批次后进行解析和去重；成功解析的交易经过大模型语义初分，高置信度结果直接入库，低置信度交易进入人工复核队列；用户完成校正后，交易数据进入 Dashboard 聚合分析，随后流向消费人格与财务健康分析模块；在报表生成环节，系统根据是否选择家庭组织参数，分别生成个人 PDF 报表或家庭聚合 PDF 报表。

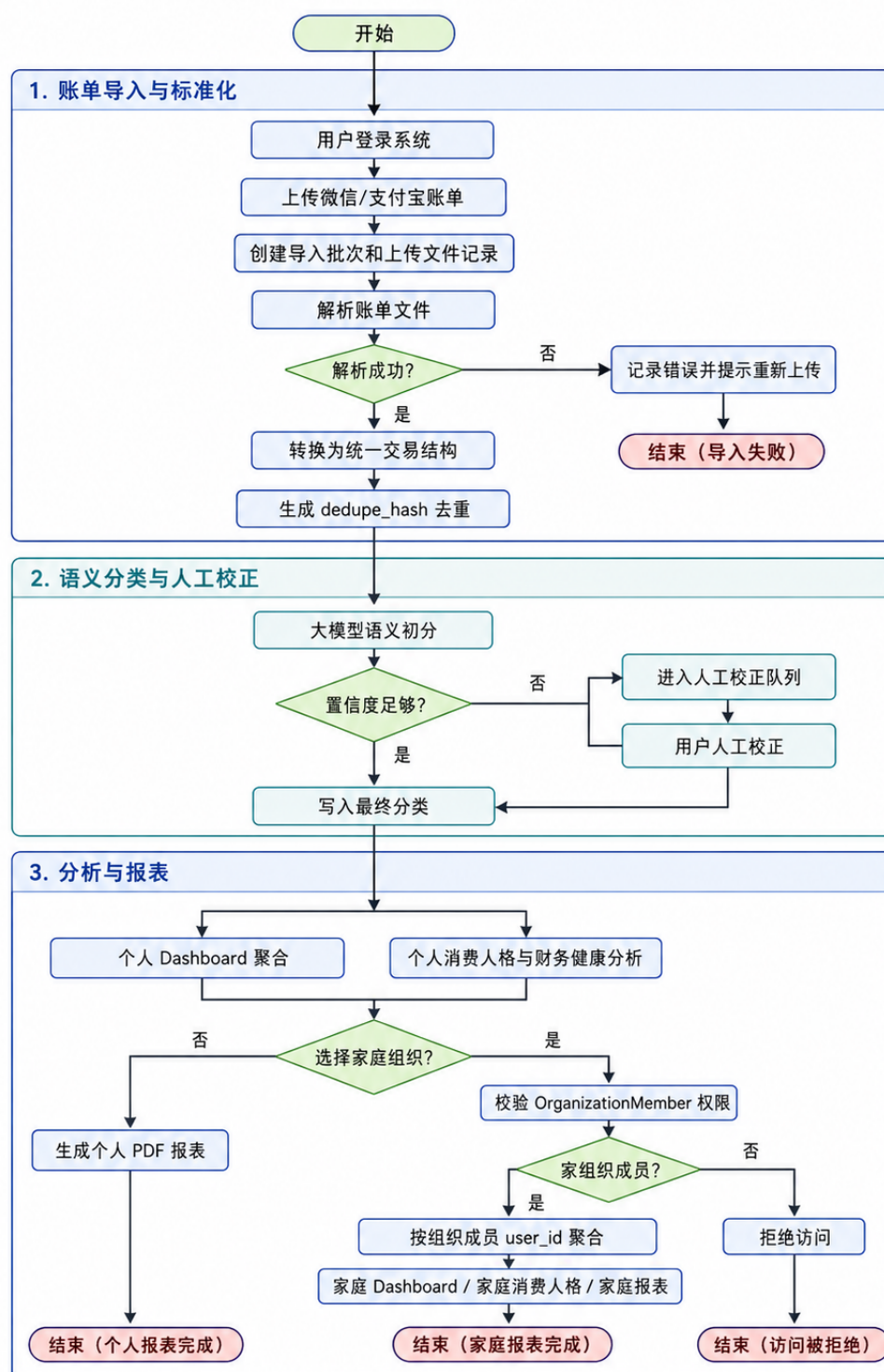


图 3.2 核心业务活动图

第四章 用户界面与体验设计

4.1 设计迭代过程

系统 UI 设计随开发迭代同步演进，经历以下三个阶段。

第一阶段——场景驱动草图：需求分析阶段，团队依据普通用户、家庭管理员、运维人员三类场景，推演各页面的核心信息与操作动线，确定整体布局框架：深色侧边栏承载全局导航，白色卡片区承载数据展示。各功能页面的布局模式在此阶段确定：Dashboard 采用摘要卡片组加图表区的双层结构；校正工作台采用左侧队列加右侧详情的双栏分割；交易列表采用顶部筛选栏加分页表格。

第二阶段——设计系统建立：首个开发迭代中，以 Dashboard 页面为标准件，确定色板、字号层级与间距规范，检验组件复用与色彩层级的一致性，形成贯穿全系统的设计语言（详见下节“设计系统”）。

第三阶段——联调反馈与细节调整：前后端联调过程中，交互细节根据真实操作反馈迭代调整。导入页面的进度展示方式、校正工作台的双栏布局与队列排序逻辑、家庭组织页面的角色权限控件可见性，均在多轮联调中根据反馈完善，最终形成本章所展示的页面形态。

4.2 设计系统

系统采用专业金融工具风格的设计体系，以深色导航与浅色内容区形成清晰的视觉层级。设计系统的核心参数如下：

表 4.1 设计系统色板

色板角色	色值	应用位置
深色侧边栏底色	#06233D	左侧导航栏背景
深色侧边栏悬浮色	#041A2E	导航项悬停/选中态
品牌主色	#00A884	主要按钮、链接、数据高亮
页面背景	#F5F7FA	内容区域整体背景
卡片背景	#FFFFFF	数据卡片、表格、表单容器
成功色	#22C55E	成功状态、收入标识
警告色	#F59E0B	警告提示、中等置信度
错误色	#EF4444	错误状态、支出标识
信息色	#3B82F6	信息提示、链接、进度条

深色侧边栏提供全局导航，顶部栏显示当前页面标题与用户信息区域。主内容区采用白色卡片承载数据面板、表格与表单，通过规则间距与统一圆角保持视觉一致性。图表采用 ECharts 渲染，配色与设计系统色板保持一致，确保图表与页面风格协调。

4.3 主要页面原型

系统共设计 10 个主要页面，本节逐一描述各页面的功能定位与关键交互。

4.3.1 登录与注册页面

登录与注册页面采用左右分栏布局：左侧为深色品牌展示区，包含应用名称与简短说明；右侧为白色表单卡片，包含用户名与密码输入框，以及登录与注册模式切换按钮。注册模式额外显示邮箱字段。验证成功后，用户进入系统主界面。

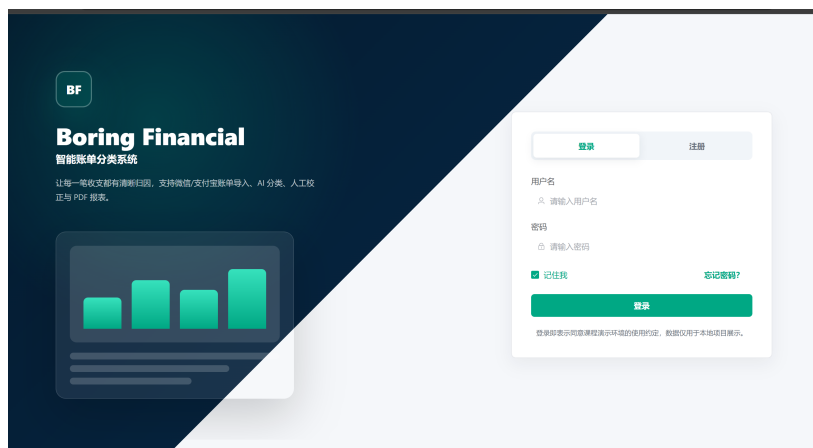


图 4.1 登录与注册页面截图

4.3.2 仪表盘页面

Dashboard 是系统的财务驾驶舱，顶部展示收入、支出、结余、储蓄率四张摘要卡片。中部为支出趋势折线图和分类占比饼图，右侧展示 Top 商户排行榜。用户可通过顶部日期筛选器切换统计周期，从整体趋势快速定位异常支出。

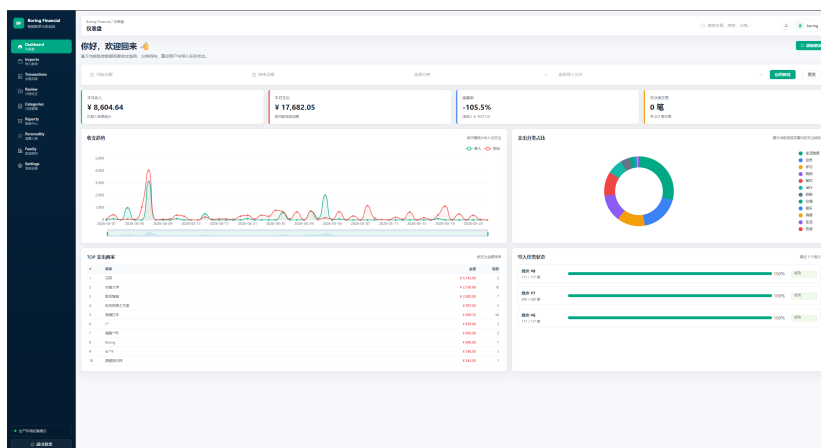


图 4.2 仪表盘页面截图

4.3.3 账单导入页面

账单导入页面提供拖拽上传区域，支持微信与支付宝 CSV 账单文件。上传成功后，页面展示最近导入记录列表，包含文件名、平台、状态与处理进度条，支持批量进度轮询。用户可点击导入记录查看详情，或删除历史导入批次。

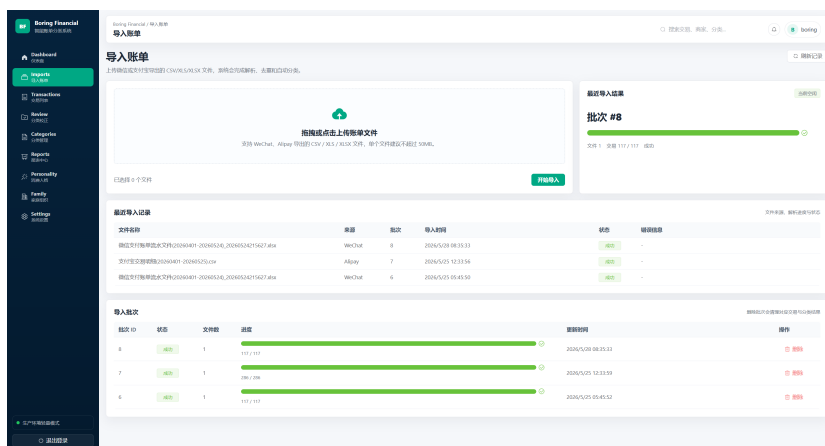


图 4.3 账单导入页面截图

4.3.4 交易列表页面

交易列表页面以筛选栏加分页表格的形式展示所有交易记录。筛选条件包括日期范围、平台（微信/支付宝）、分类、导入文件和关键词搜索。表格每行显示时间、商户、金额、分类与置信度。点击行可展开详情抽屉，展示分类来源、置信度、分类理由与原始交易备注。

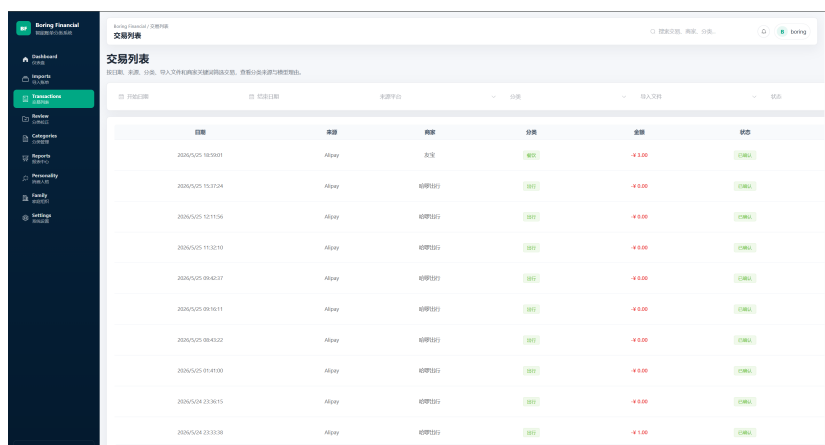


图 4.4 交易列表页面截图

4.3.5 分类校正工作台

校正工作台采用左右分栏布局：左侧为待校正交易队列，按置信度从低到高排序，每项显示商户、金额与当前分类；右侧为当前选中交易的详细信息，包括模型分类建议（来源、置信度、推理理由），以及分类选择控件，允许用户选择新的系统分类或自定义分类并确认修正。确认后交易从队列中移除。

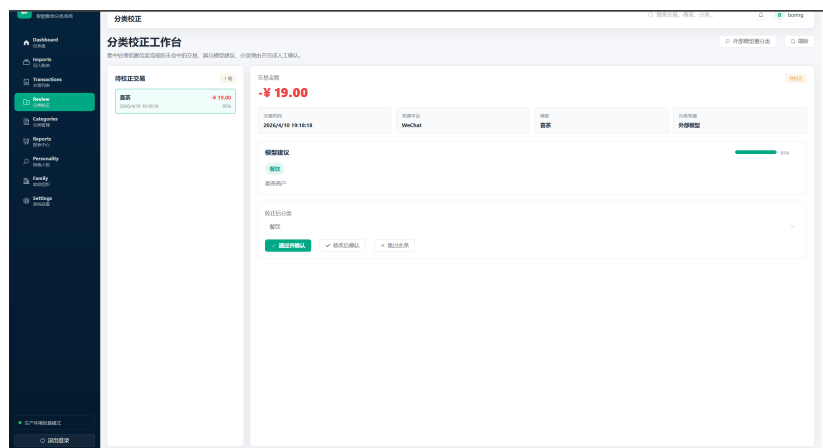


图 4.5 分类校正工作台截图

4.3.6 分类管理页面

分类管理页面展示系统预设分类与用户自定义分类。顶部为分类使用频率统计，按交易数量排序。中部为分类列表，系统分类标记为锁定不可删除，用户自定义分类支持新增、编辑与禁用操作。修改操作通过弹窗表单完成，实时生效。

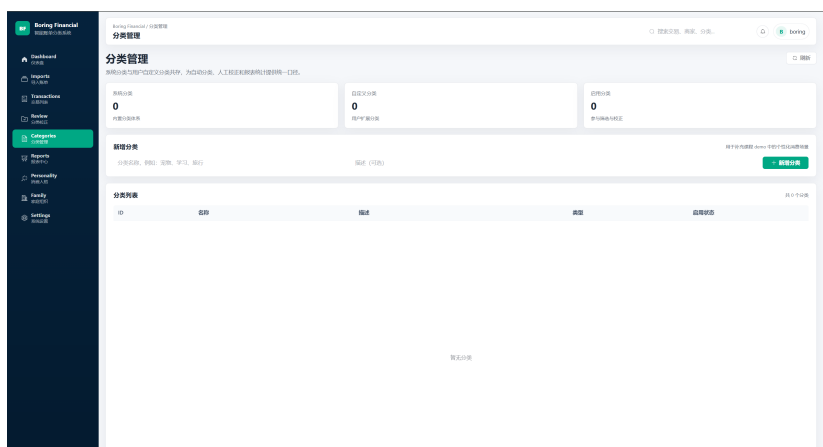


图 4.6 分类管理页面截图

4.3.7 消费人格页面

消费人格页面由三个区域组成：顶部为 16 型消费人格画像卡片，展示用户所属人格类型及简要描述；中部为四维雷达图（现时偏好、心理账户、炫耀性消费、开放性），展示各维度得分；底部为财务健康评分区域，展示储蓄率、收入稳定性、消费多样性、支出波动性和应急能力的子指标。页面同时提供问卷自评入口，用户可对比主观认知与交易数据画像的偏差。

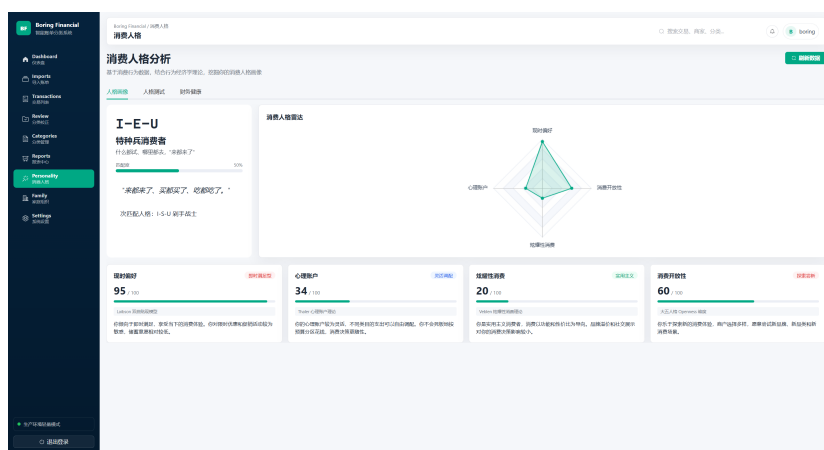


图 4.7 消费人格页面截图

4.3.8 家庭组织页面

家庭组织页面在用户未加入任何组织时显示创建引导。已有组织时，页面展示三部分内容：左侧为成员列表，每行显示用户名、角色与管理按钮；右上方为家庭财务概览卡片，展示组织合计收入、支出与交易数；右下方为家庭消费人格聚合雷达图，将所有成员交易数据按人格维度聚合展示。家庭所有者和管理员可见角色管理控件与重分类触发按钮。

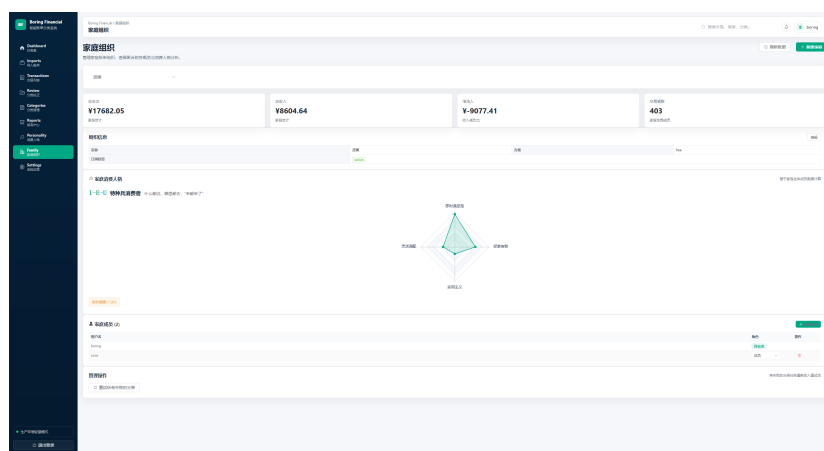


图 4.8 家庭组织页面截图

4.3.9 报表中心页面

报表中心页面提供日期范围选择器与导入文件筛选器。用户设置条件后点击生成按钮，系统异步创建报表任务。生成完成后，页面展示报表摘要信息（覆盖时间段、交易数、分类分布），并提供 PDF 下载按钮。历史报表列表按生成时间倒序排列，支持重新下载。

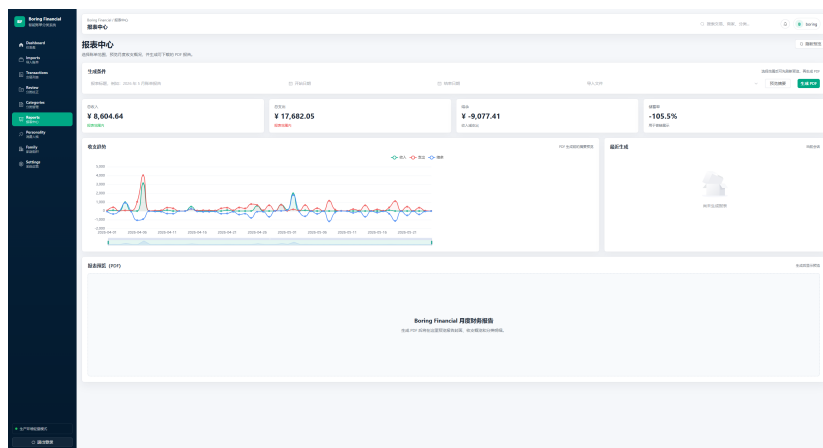


图 4.9 报表中心页面截图

4.3.10 系统设置页面

系统设置页面向所有用户展示当前分类供应商名称、置信度阈值等配置信息。管理员用户额外可见重试队列控制区：包括当前队列状态（等待重试数、重试中数、已失败数），以及“一键重试历史超时”按钮。普通用户仅可见配置展示，管理操作控件隐藏。

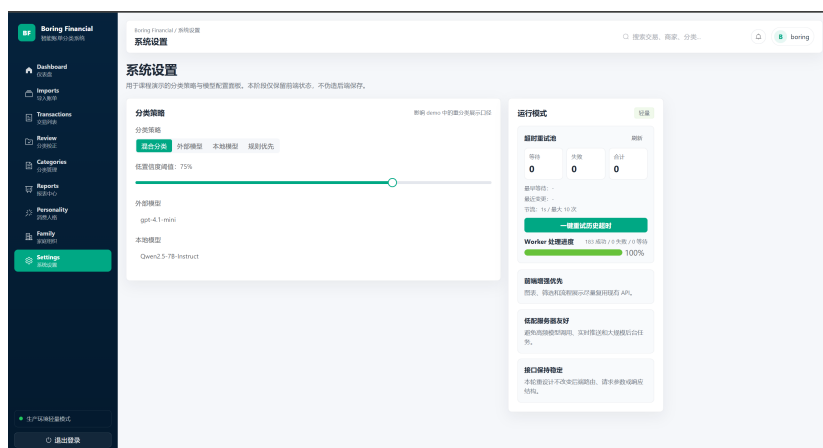


图 4.10 系统设置页面截图

4.4 界面信息架构与交互组织

系统界面围绕“全局导航、任务页面、状态反馈、数据可视化”四类信息组织。该划分使用户在不同业务页面之间切换时保持一致的认知模型，也便于后续新增功能时复用相同的交互规则。

表 4.2 界面信息架构与交互组织

界面区域	设计职责
全局导航	以深色侧边栏承载主要功能入口，保证用户可在导入、交易、校正、分析、报表等任务之间快速切换
页面标题与用户区	在顶部区域展示当前任务语境、用户身份和必要操作，减少用户对当前位置的判断成本
任务主区域	各页面按“筛选—列表—详情”或“概览—图表—明细”的结构组织内容，匹配财务数据查看和校正流程
状态反馈	导入、报表生成和模型重试等耗时任务通过进度、状态标签和结果摘要展示处理进展
权限可见性	管理员功能、家庭角色管理和普通用户配置展示在界面上区分可见，避免误操作
数据可视化	看板、人格画像和家庭聚合页面采用图表表达趋势、占比和维度评分，辅助用户理解财务状态

交互组织上，系统将身份状态、当前组织和页面筛选条件保持为跨页面可感知的上下文。用户切换页面后不需要重复登录或重新理解导航结构；需要管理员权限的操作仅在具备权限时出现，普通用户界面保持简洁。

4.5 用户体验目标

系统用户体验设计围绕以下四个核心目标展开：

减少用户操作步骤：账单导入采用拖拽上传模式，一次上传即可触发解析 → 去重 → 分类的全自动链路。Dashboard 与人格页面的数据通过后端聚合计算，用户无需手动维护分类映射表或公式。批量校正工作台允许用户在单一页面完成所有低置信度交易的复核，无需逐条跳转。

分类结果可解释、可校正：模型分类结果不仅给出类别，还提供置信度与人类可读的推理理由。校正工作台将模型建议与修改控件并置，用户可在了解 AI 推理过程后做出校正决策。校正结果即时生效，无需额外保存步骤。

校正数据全链路贯通：人工校正后的交易分类应实时反映在 Dashboard 聚合、消

费人格计算和 PDF 报表中。系统通过统一数据源（transactions 表）确保各分析模块使用一致的最新分类结果，避免出现“已校正但图表未更新”的不一致情况。

个人模式不受家庭功能影响：家庭组织功能采用显式 opt-in 设计——用户需主动创建或加入组织，并在看板和报表页面主动切换至家庭视图。未切换时，所有页面保持个人账本模式，确保现有用户的体验不受影响。

第五章 系统设计

本章阐述 Boring Financial 智能账单分类系统的架构设计，涵盖总体架构、技术选型、前后端分层设计、数据隔离策略、部署方案以及安全机制。

5.1 总体架构

系统采用四层架构，自上而下分为展示层、网关层、服务层与数据/外部服务层，各层通过 HTTP 协议通信，职责边界清晰。

1. **展示层**: Vue 3 + Element Plus + ECharts 构建的单页应用，运行于用户浏览器，负责全部用户交互与数据可视化。
2. **网关层**: 生产环境使用 Nginx 对外提供静态文件服务，并通过反向代理将 /api 请求转发至后端。开发环境使用 Vite Proxy 实现同源转发。
3. **服务层**: FastAPI 构建的 RESTful API 服务，包含认证、导入、交易、分类、Dashboard 聚合、消费人格、家庭组织及 PDF 报表等业务模块。异步任务通过 Celery Worker 消费，外部模型调用通过 Retry Queue Worker 后台线程串行管理。
4. **数据与外部服务层**: PostgreSQL (生产) / SQLite (轻量部署) 作为主数据库, Redis 作为 Celery 消息代理。外部服务包括 OpenAI-compatible API 与本地模型服务 (vLLM)，由 Retry Queue Worker 统一调度。

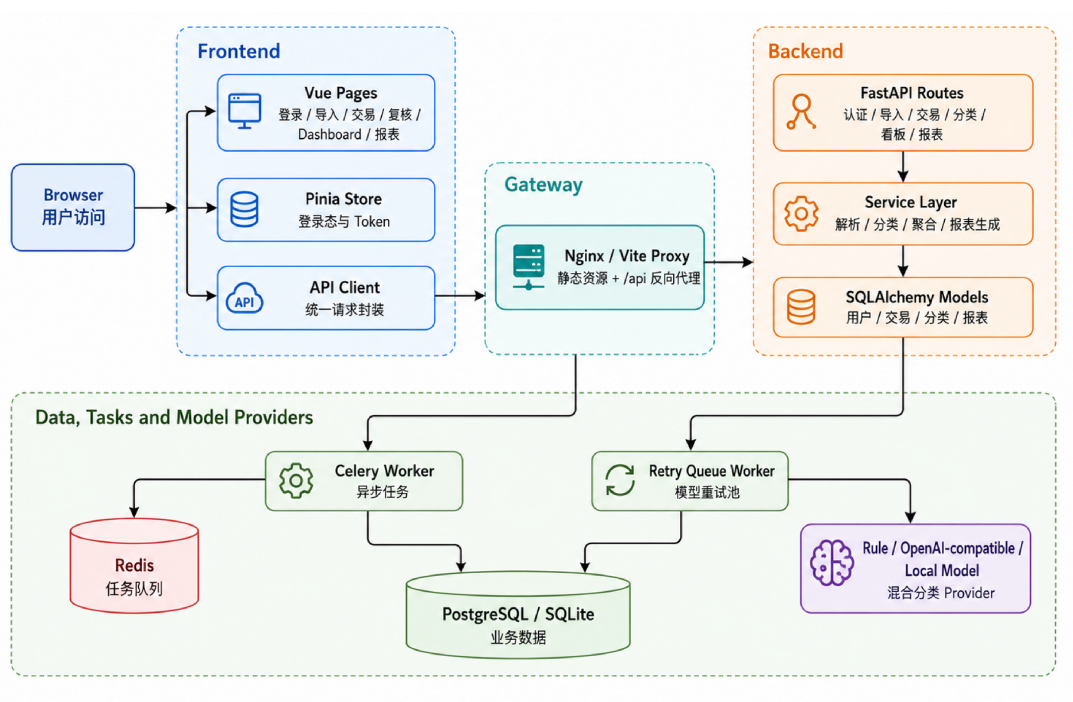


图 5.1 系统总体架构图

5.2 技术选型

下表列出各层采用的核心技术及其选型依据。

表 5.1 技术选型汇总

技术	用途	选型依据
FastAPI (Python)	后端 API 框架	原生异步支持，自动生成 OpenAPI 文档 (/docs、/redoc)，Pydantic 请求/响应验证
Vue 3 + Type-Script	前端框架	组合式 API 支持高复用的逻辑组织，Type-Script 编译期类型检查降低运行时错误
Element Plus	UI 组件库	成熟的 Vue 3 组件生态，提供表格、表单、弹窗、导航等企业级组件
ECharts	数据可视化	支持雷达图、柱状图、折线图、饼图等，满足 Dashboard 与人格分析的可视化需求
SQLAlchemy 2.x	ORM 框架	Data Mapper 模式解耦领域逻辑与持久化，支持 SQLite/PostgreSQL 双数据库切换
Celery + Redis	异步任务队列	Celery 提供生产级任务调度与重试，Redis 作为轻量消息代理兼缓存
fpdf2	PDF 生成	轻量级 Python PDF 库，无外部依赖，支持 Unicode 中文字体加载
Docker Compose	容器编排	一键启动多服务栈，确保开发/测试/生产环境一致性

5.3 前端架构

前端代码按职责划分为四层：视图层由 10 个页面组件构成，对应登录、Dashboard、导入、交易列表、校正工作台、分类管理、消费人格、家庭组织、报表中心和系统设置；状态层通过 Pinia Store 管理认证状态与 Token 持久化；通信层基于 Axios 封装 API 客户端，通过请求拦截器自动注入 Bearer Token；路由层使用 Vue Router 配置页面懒加载与导航守卫，未登录用户自动重定向至登录页。类型定义层为全部业务模型提供 TypeScript 接口约束。

前端视觉设计规范以深色导航（#06233D）、品牌色（#00A884）与浅灰页面背景（#F5F7FA）构成统一视觉语言，详见第 4 章。

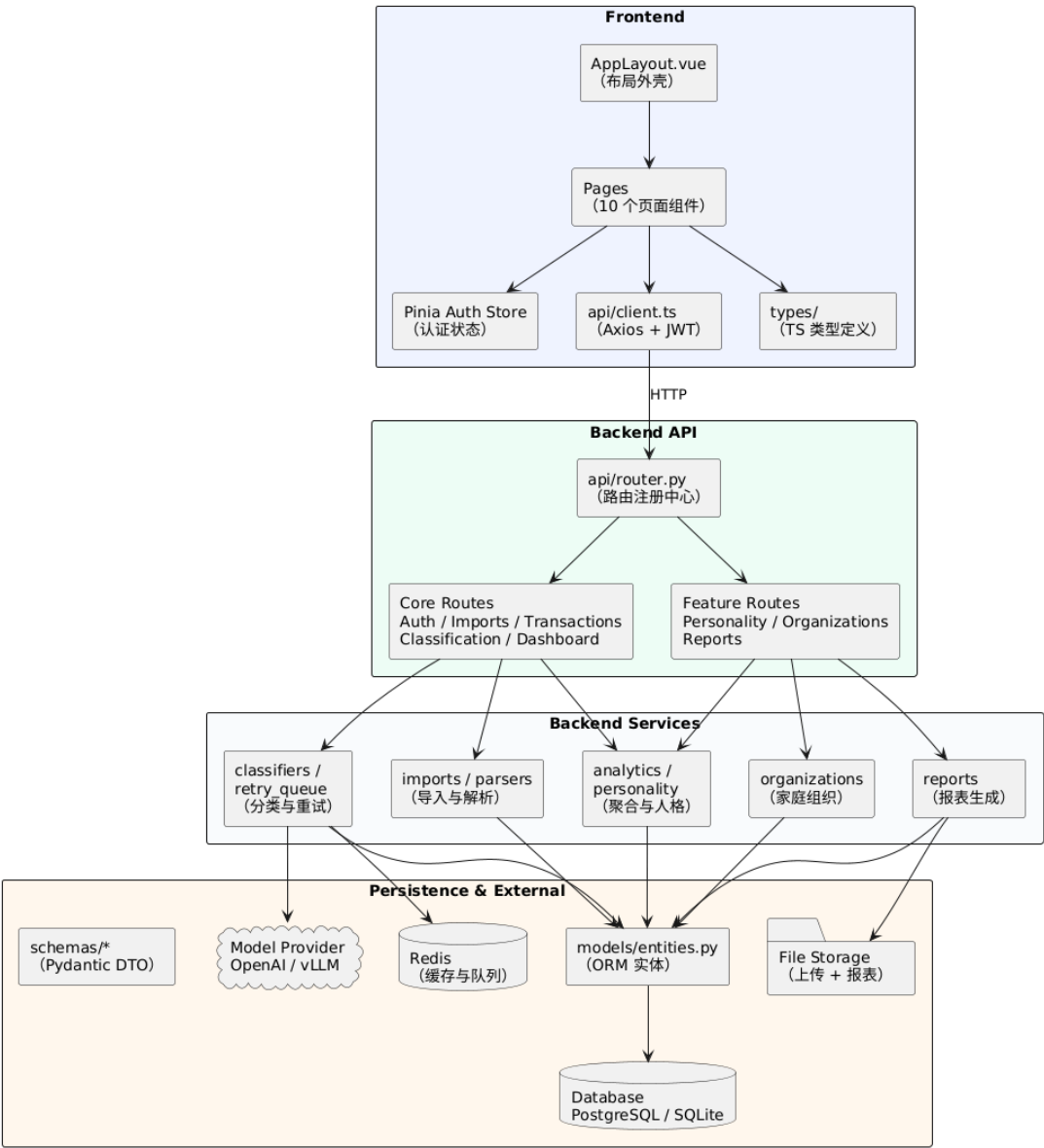


图 5.2 UML 组件图

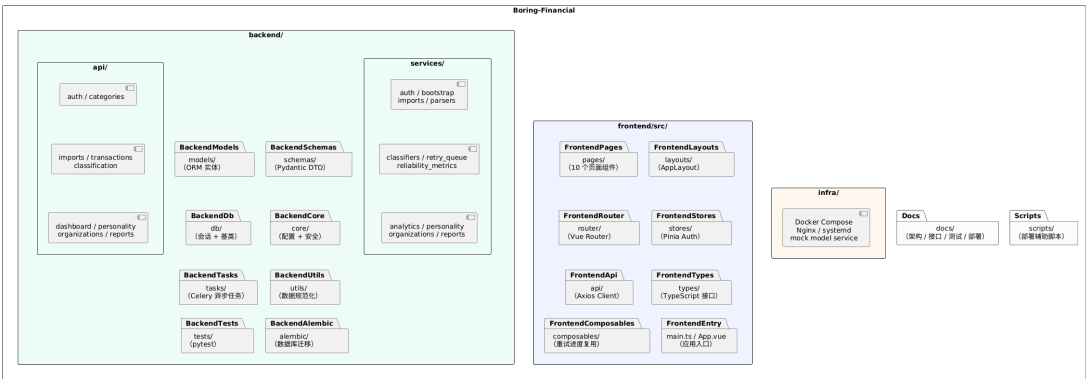


图 5.3 系统包图

5.4 后端架构

后端按分层架构组织为四层。路由层 (api/) 通过 router.py 统一注册 10 个路由模块, 覆盖认证、分类、导入、交易、分类校正、Dashboard、人格分析、家庭组织和报表生成。服务层 (services/) 承载全部业务逻辑: imports.py 管理导入批次与文件处理, parsers.py 实现多平台账单解析与规范化, classifiers.py 以复合分类链路编排规则、缓存与模型调用, retry_queue.py 提供后台串行重试池, analytics.py 与 personality.py 负责聚合计算与人格分析, organizations.py 实现家庭组织 CRUD 与角色校验, reports.py 以 Builder 模式构建 PDF 报表。数据层(models/entities.py) 定义 12 个 SQLAlchemy ORM 实体, schemas/ 按业务域拆分的 Pydantic DTO 负责请求校验与响应序列化。基础设施层(db/、core/、tasks/) 提供数据库会话管理、JWT 安全模块、配置加载与 Celery 异步任务定义。

5.5 数据隔离策略

系统通过以下机制保障多用户数据隔离与家庭组织聚合访问:

1. **个人资源归属**: 所有用户私有资源 (交易、导入批次、上传文件、分类缓存、报表等) 通过 user_id 外键归属。系统分类的 user_id 为 NULL, 用户自定义分类的 user_id 指向创建者。
2. **查询过滤**: 业务接口通过 Bearer Token 提取当前用户身份, 查询时强制附加 WHERE user_id = current_user.id 条件。
3. **家庭聚合模式**: 家庭组织不改变交易归属。用户选择家庭视图时, 前端传入可选 organization_id。后端先通过 OrganizationService 校验成员身份, 再收集组织内全部成员 user_id 列表, 传入 Dashboard、Personality 或 Reports 服务实现跨成员数据聚合。
4. **默认模式**: organization_id 为可选参数, 不传入时各接口按个人账本口径返回数据, 保持向后兼容。

5.6 部署架构

系统支持两种部署模式以适应不同规模的运行环境:

轻量本地部署 使用 SQLite 作为数据库, TASK_ALWAYS_EAGER = true 使 Celery 任务在当前进程同步执行, Retry Queue Worker 在后台线程运行。前端通过 Vite 开发服务器或构建后的静态文件直接访问。

生产环境部署 Docker Compose 编排多容器服务栈: PostgreSQL 数据库、Redis 消息代理、Nginx 反向代理 + 静态文件服务、FastAPI/Uvicorn (监听 127.0.0.1:8000)、Celery Worker、Retry Queue Worker 后台线程。外部模型服务作为独立外部依赖。

通信链路为：用户浏览器 → Nginx（静态资源直接返回，API 请求反向代理至 FastAPI）→ FastAPI 与 PostgreSQL、Redis、Retry Queue Worker 通过内部网络通信。上传文件与 PDF 报表存储于 storage/ 目录。

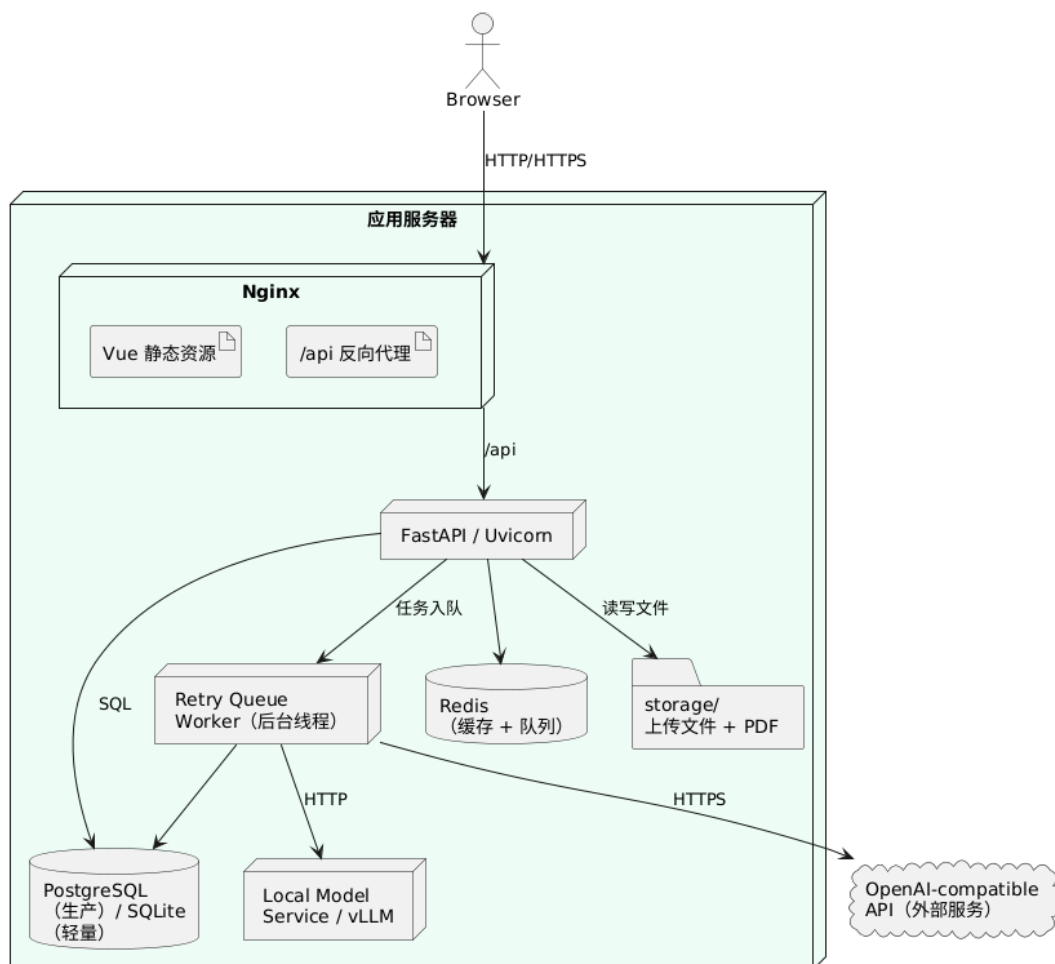


图 5.4 系统部署图

5.7 安全机制

系统的安全设计覆盖认证、授权、数据隔离、注入防护与隐私保护五个层面。

表 5.2 安全机制汇总

安全风险	应对措施	覆盖层面	说明
身份伪造	密码哈希与短期访问凭证	认证层	密码不存明文；登录凭证具有有效期
未授权访问	业务请求强制验证用户身份	接口入口	未登录或凭证无效的请求被拒绝
跨用户数据泄露	私有资源查询强制按用户归属过滤	服务层与数据层	用户只能访问自己的交易、分类和报表
组织越权操作	家庭组织角色权限控制	组织管理模块	所有者和管理员可管理成员，普通成员仅查看
SQL 注入	参数化查询与统一数据访问	数据访问层	避免通过字符串拼接构造查询
恶意文件上传	文件大小限制、扩展名白名单	导入模块	只接受受支持账单格式
模型隐私	可切换外部模型、本地模型或规则兜底	分类模块	已识别外部调用风险，并保留本地化方案

安全模块将身份校验、管理员校验和资源归属检查从具体业务流程中抽离，作为统一的横切能力复用。此设计使安全策略变更不影响账单导入、分类校正和报表生成等核心业务逻辑。

第六章 程序开发

本章阐述系统的程序组织方式、数据库设计、设计模式应用以及核心流程设计。重点说明这些设计如何支撑多平台账单解析、语义分类、人工校正、家庭聚合与报表生成，而不是逐行展开具体代码。

6.1 面向对象程序组织

系统并非全量采用面向对象范式。对于认证、接口编排和简单聚合计算，过程式组织更直接；对于账单解析、分类策略、领域实体和报表生成等存在扩展变化点的部分，则采用面向对象方法控制复杂度。

6.1.1 领域实体及其关联关系

系统使用 ORM 建立领域实体与数据库表之间的映射，采用 Data Mapper 模式将领域概念与持久化细节解耦。实体对象主要表达用户、账单、交易、分类、组织和报表之间的关系，查询与事务处理则由服务层统一承担。

对于创建时间、更新时间等跨实体通用属性，系统采用 Mixin 方式复用公共字段，避免在多个实体中重复维护相同定义。该设计降低了后续调整审计字段时的维护成本。

实体关联的核心拓扑以用户为中心展开：用户拥有导入任务、交易记录、自定义分类和报表；导入批次关联原始上传文件与解析后的交易；交易同时保留系统建议分类和用户最终确认分类，以支持“AI 初分 + 人工校正”的业务闭环；家庭组织通过成员关系连接多个用户，在不改变个人账本归属的前提下实现聚合分析。

交易实体的关键设计是去重、分类追溯和最终口径分离。系统通过交易特征生成稳定指纹来识别重复导入；建议分类与最终分类分开保存，使模型判断和人工修正可以并存；分类历史以追加方式记录，便于后续分析分类策略变化。

6.1.2 分类器策略

分类子系统围绕策略、责任链与外观三种模式组织，用于在规则分类、缓存分类、外部大模型和本地模型之间建立可替换、可降级的分类链路。

外观对象对外提供统一分类入口，内部按“缓存优先、模型判断、规则兜底”的顺序组织责任链。模型服务不可用或超时，系统不会阻塞整批导入，而是将交易转入后续重试流程；规则分类作为最低保障，确保系统在外部依赖异常时仍能给出基础结果。外部模型与本地模型遵循同一输出约定，因此后续更换模型供应商时不需要改动上层业务流程。

Strategy 模式使不同分类策略可以独立演进；Chain of Responsibility 提供清晰的降级路径；Facade 模式隐藏缓存、重试和结果记录等内部复杂性，使导入流程只需关心“获得分类建议”这一业务结果。

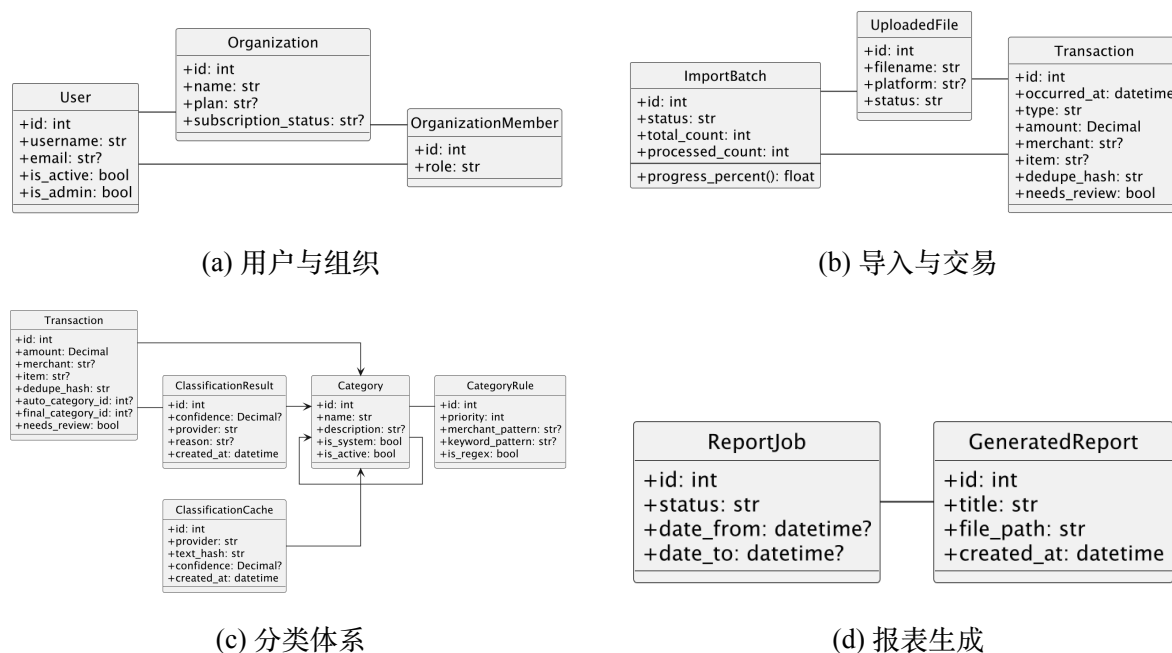


图 6.1 领域实体类图

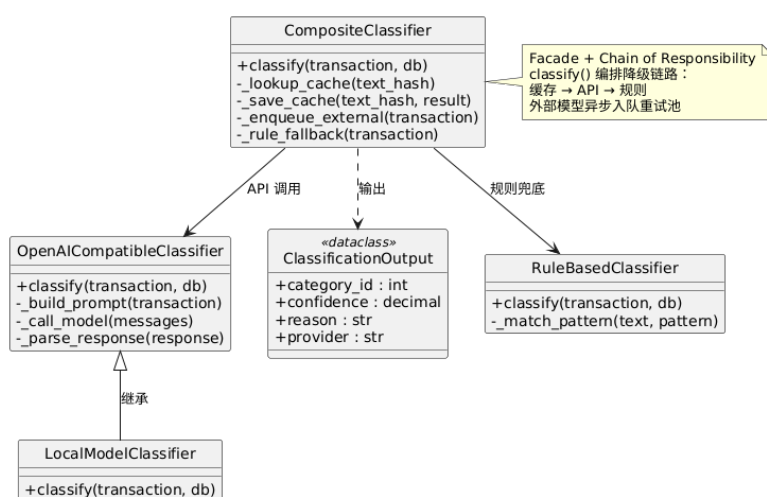


图 6.2 分类器策略类图

6.1.3 账单解析器

解析子系统负责将微信/支付宝异构账单文件转换为统一交易结构，采用策略、适配器与注册表三种模式。

解析子系统先识别账单来源，再选择对应解析策略，将不同平台的列名、编码和表头差异统一转换为系统内部交易格式。读取器负责屏蔽文件编码和表格格式差异，解析器负责平台语义映射，规范化环节负责生成稳定的交易特征。

支付宝与微信账单的差异被封装在各自解析策略中，上层导入流程只处理统一交易结果。Strategy 模式支持新增平台时局部扩展；Adapter 模式把底层文件读取复杂性收敛为统一读取能力；Registry/Factory 模式负责解析器选择，避免在业务流程中堆积平台判断逻辑。

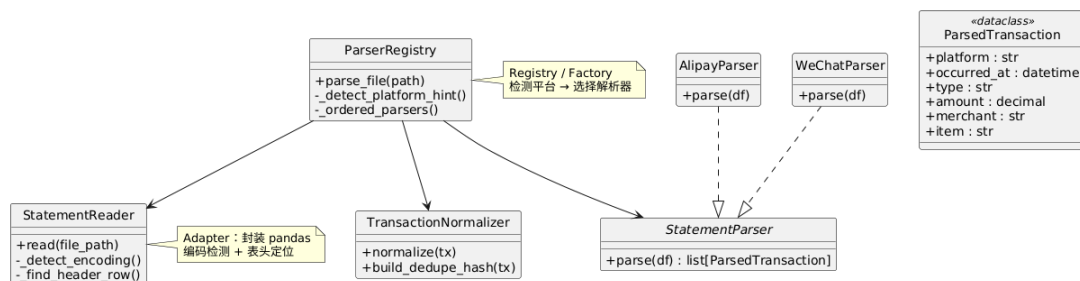


图 6.3 账单解析器类图

6.1.4 报表构建器

报表生成采用 Builder 模式，将复杂 PDF 构建过程拆分为封面、摘要指标、分类汇总、商户排行和交易明细等有序步骤。

构建过程先准备统计上下文，再按固定顺序渲染各章节，最后保存报表并记录生成结果。数据聚合与版面渲染分离，使同一份统计数据可以服务于不同报表页面，也便于后续加入新的分析章节。

Builder 模式使每个构建步骤关注单一报表片段，新增章节时只需扩展构建序列，不会影响导入、分类和看板等其他业务模块。

6.2 数据库设计

系统数据库共 12 张表，按业务域分两组展示，分别对应核心账单分类闭环和家庭组织、报表等扩展能力。

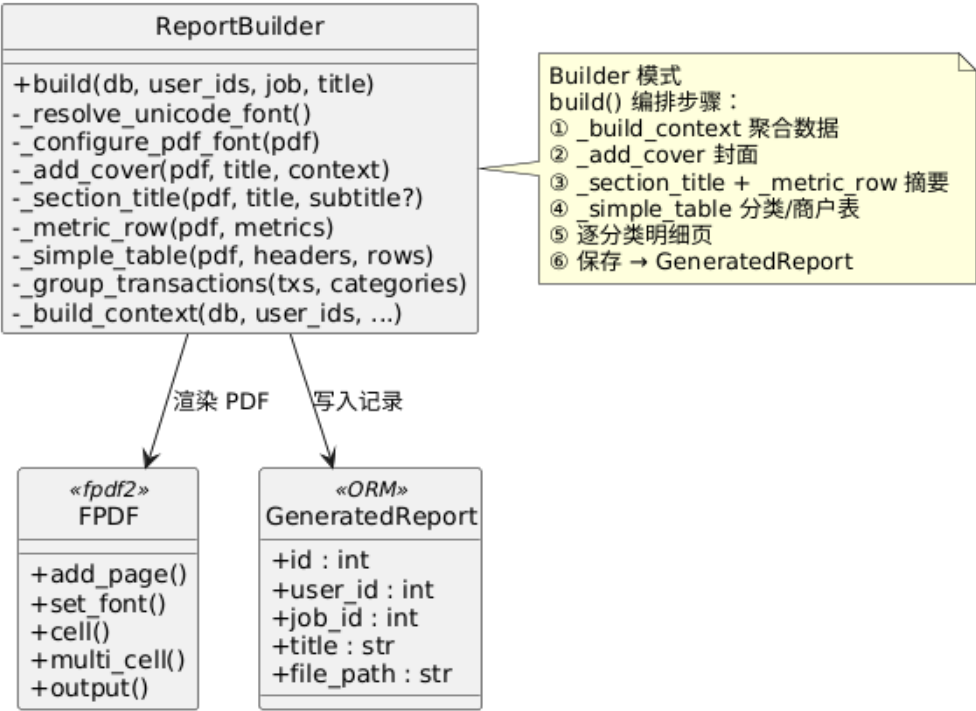


图 6.4 报表构建器类图

6.2.1 核心账单与分类

核心域涵盖用户、分类、分类规则、导入批次、上传文件、交易、分类结果和分类缓存八类数据对象。

关键设计决策包括：按用户维度识别重复交易，避免同一账单被反复导入；同时保存建议分类和最终分类，保证人工校正优先于自动判断；低置信度交易被纳入校正队列；分类历史采用追加记录，支持追溯模型表现；分类缓存按用户和供应商隔离，避免跨用户数据混淆。

6.2.2 家庭组织与报表扩展

扩展域涵盖家庭组织、组织成员、报表任务和已生成报表四类数据对象，并复用核心域中的用户与交易数据。

关键设计决策是家庭组织不改变个人交易归属，而是在查询时按成员范围聚合数据。家庭所有者、管理员和普通成员拥有不同操作权限；报表任务独立记录生成状态，已生成报表独立记录输出结果，便于用户重复下载和运维人员排查失败任务。

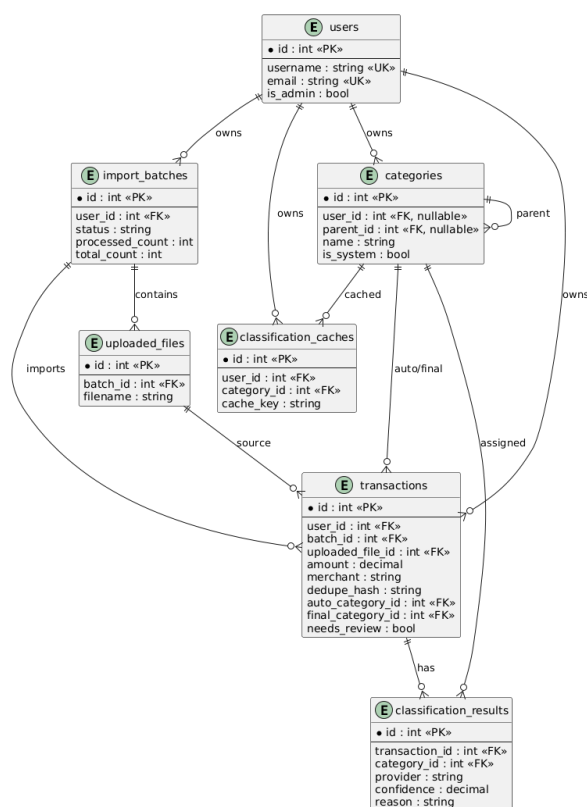


图 6.5 核心账单与分类 ER 图

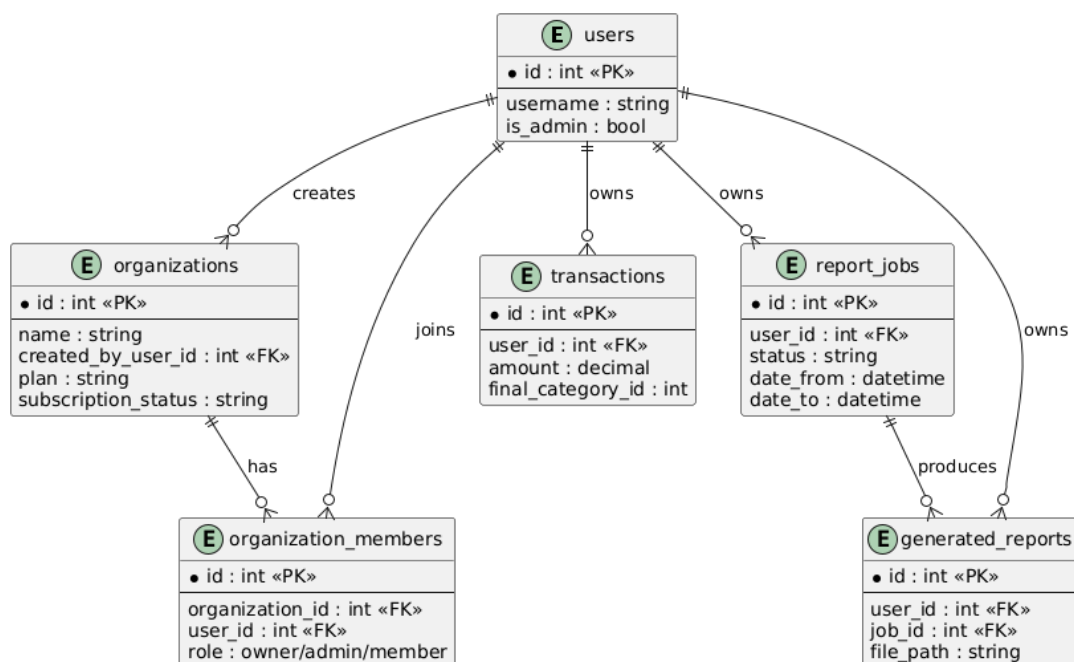


图 6.6 家庭组织与报表扩展 ER 图

6.3 设计模式应用

下表汇总系统在设计模式层面的应用情况，便于教师在单一位置审视 OO 设计的完整性。

表 6.1 设计模式应用汇总

设计模式	应用区域	设计作用	对应节
Data Mapper	领域实体与持久化层	将业务实体与数据库访问解耦，降低数据层变化对业务层的影响	6.1.1
Mixin	通用审计字段	复用创建时间、更新时间等公共属性，减少重复定义	6.1.1
Strategy	分类器与平台解析器	封装可替换的分类和解析策略，支持供应商或平台扩展	6.1.2/6.1.3
Chain of Responsibility	分类链路	分类请求沿缓存、模型、规则顺序处理，不可用时自动降级	6.1.2
Facade	分类入口	封装缓存、模型调用、重试和规则兜底，对导入流程暴露统一能力	6.1.2
Adapter	账单文件读取	将多编码、多格式账单文件适配为统一读取结果	6.1.3
Registry/Factory	解析器选择	根据文件特征选择合适解析策略，避免业务流程耦合平台判断	6.1.3
Builder	PDF 报表生成	将多页报表拆分为封面、摘要、表格、明细等构建步骤	6.1.4
Template Method	模型供应商适配	固定模型分类的主要步骤，使不同供应商只替换差异部分	6.1.2
DTO	接口数据契约	区分外部输入输出与内部实体，提升接口可维护性	5.7
Dependency Injection	认证与会话管理	将身份校验、数据库会话等横切能力从业务逻辑中抽离	5.7
Producer-Consumer	模型重试队列	将外部模型调用从用户请求中解耦，提升故障恢复能力	8.3
RBAC	家庭组织权限	按角色区分查看、管理和成员维护权限，降低越权风险	5.5

6.4 核心流程设计

6.4.1 账单导入与大模型初分

用户上传账单文件的完整处理链路包含六个阶段。(1) 用户提交账单文件后，系统创建导入任务并向页面反馈处理进度。(2) 系统识别账单平台，选择对应解析策略，将不同格式转换为统一交易结构。(3) 交易进入规范化与去重环节，重复记录被跳过，新交易进入分类流程。(4) 分类链路按缓存、模型、规则的顺序生成分类建议，外部模型异常时转入重试机制。(5) 系统保存分类建议、置信度和理由，同时记录分类历史，便于后续追溯。(6) 导入任务结束后，页面展示成功、失败和待复核交易数量，引导用户进入校正或分析页面。

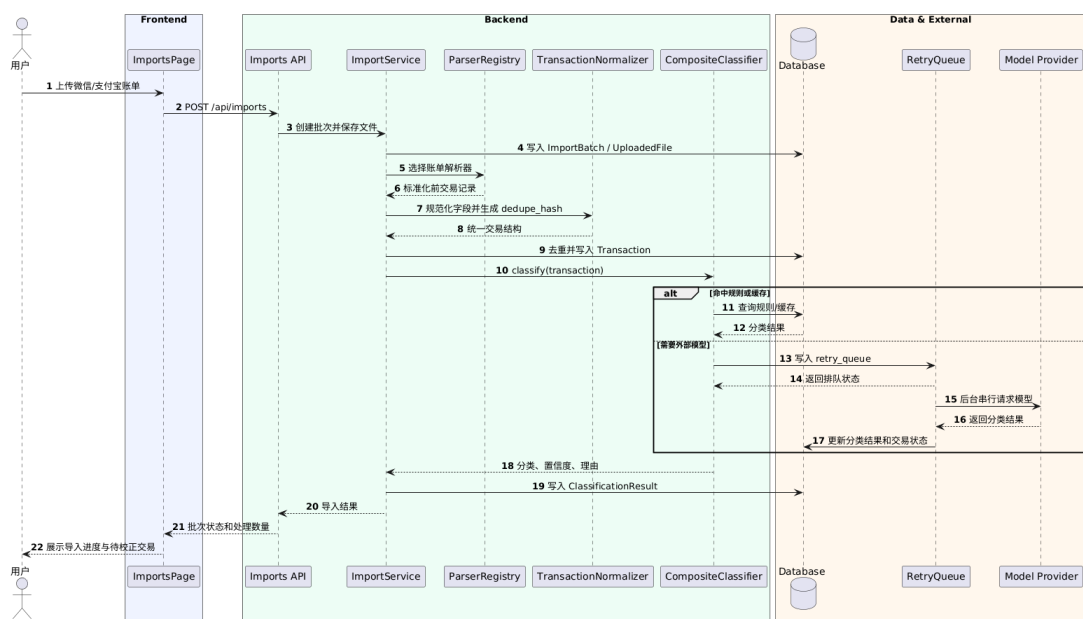


图 6.7 账单导入与大模型初分顺序图

6.4.2 家庭组织聚合查询

家庭组织视图的数据聚合不改变交易归属，而是在查询层面根据成员身份扩大统计范围。用户进入组织页面后，系统展示其加入的组织、角色和成员信息。查看家庭看板、人格画像或报表时，系统先确认用户属于该组织，再汇总组织成员的交易数据。看板返回家庭收入、支出、净流入和分类分布，人格分析基于成员交易计算家庭整体画像，报表模块生成家庭口径的 PDF 文档。

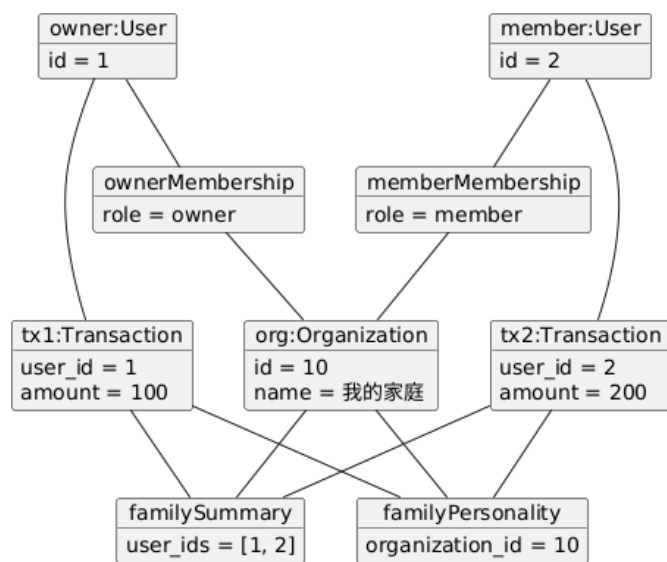


图 6.8 家庭聚合场景对象图

第七章 验收与发布

7.1 自动化测试

7.1.1 静态代码验证

在动态测试之前，团队采用了两类静态验证手段。

TypeScript 类型检查：前端 build 脚本将 `vue-tsc --noEmit` 配置为必要前置步骤，任何类型不兼容错误均会阻断构建流程。该检查覆盖全部页面组件、Pinia Store、API 接口封装与 `types/` 目录下的所有接口定义，确保前端数据结构与后端 API 契约的类型一致性。

代码走查：每个开发迭代完成后，团队成员对本迭代新增代码进行交叉走查 (code walkthrough)，重点核查认证与鉴权路径 (`get_current_user` 依赖注入、资源归属校验)、多用户数据隔离边界与分类器异常分支的覆盖情况，走查意见在迭代内即时修正。

7.1.2 动态测试

系统后端具备完整的自动化测试套件，位于 `backend/tests` 目录下，共 16 个文件 (含 `confest.py` 与辅助模块)，其中 14 个为正式测试文件，覆盖安全、认证、导入、交易、分类、解析、分析、人格、组织、报表、可靠性与重试队列等核心领域。测试框架使用 `pytest`，可通过 `pytest` 或 `pytest --cov=backend` 运行。

表 7.1 自动化测试覆盖范围

测试文件	覆盖领域	测试数
test_security.py	密码哈希、Token 编解码	2
test_api_auth.py	注册、登录、登出 API	3
test_api_imports_dashboard_reports.py	导入、Dashboard 聚合、报表 API	4
test_api_transactions.py	交易查询与分类更新 API	4
test_classifiers.py	规则匹配、缓存、复合分类器	7
test_parsers.py	微信/支付宝账单格式解析	4
test_analytics.py	Dashboard 聚合计算	1

接下页

表 7.1 自动化测试覆盖范围（续）

测试文件	覆盖领域	测试数
test_personality.py	人格维度计算、问卷对比、边界条件	8
test_organizations.py	组织创建、成员权限、家庭聚合、个人模式兼容	13
test_reports.py	PDF 报表生成	1
test_reliability_metrics.py	分类可靠性指标统计	3
test_retry_queue.py	重试队列入队、处理与状态查询	4
test_normalizers.py	文本规范化与哈希稳定性	2
test_imports.py	导入服务流程	3

7.2 测试结果

全部 59 个测试用例通过，后端整体覆盖率为 75%。各测试文件覆盖其对应服务或路由模块的核心逻辑路径。

前端除静态类型检查外，还建立了基于 Vitest + happy-dom 的单元测试体系。测试套件位于 frontend/src/__tests__/ 目录下，覆盖认证状态管理（Pinia Store 的 token 持久化、login/logout 流程）、API 客户端拦截器（Authorization Header 注入逻辑）、路由守卫（未登录重定向、已登录跳转、fetchMe 调用时机）以及重试进度 Composable（SSE 流解析、百分比计算、异常恢复）。运行命令为 npm run test，全部 26 个测试用例通过。

前端构建测试包含两个阶段：TypeScript 类型检查（vue-tsc --noEmit）与 Vite 生产构建（vite build）。验证时通过 npm ci 安装依赖后执行 npm run build，两个阶段均通过，无类型错误或编译错误。Vite 构建输出提示部分 chunk 大小超过 500 KB，主要来源于 Element Plus 与 ECharts 组件库，属于后续性能优化项，不影响功能完整性。

表 7.2 测试结果汇总

测试类型	结果	说明
后端 pytest 单元/集成测试	59/59 通过	覆盖 14 个测试文件
后端代码覆盖率	75%	pytest --cov=backend 统计
前端 Vitest 单元测试	26/26 通过	覆盖 4 个测试文件
前端 TypeScript 类型检查	通过	vue-tsc --noEmit 无错误
前端 Vite 生产构建	通过	静态资源构建成功
前端 Chunk 体积优化	待优化	Element Plus + ECharts 超过 500KB

7.3 验收测试

验收测试以手工联调方式执行，覆盖从注册到报表生成的完整用户流程，共 14 个验收步骤。

表 7.3 验收测试执行清单

步骤	操作	预期结果
1	注册新用户并登录	注册成功，跳转至 Dashboard
2	进入导入页面，上传微信或支付宝账单	文件上传成功，显示导入批次进度
3	检查最近导入结果与批次进度	批次状态从 processing 变为 done，交易计数正确
4	进入 Dashboard 查看概览	收入、支出、储蓄率、分类占比与 Top 商户正常显示
5	进入交易列表，按条件筛选	筛选结果正确，分页可用
6	打开交易详情抽屉	显示分类来源、置信度与分类理由
7	进入校正工作台，确认一笔待校正交易	交易从待校正队列移除，最终分类已更新

接下页

表 7.3 验收测试执行清单（续）

步骤	操作	预期结果
8	新增一个用户自定义分类	分类列表中显示新分类，校正工作台可选
9	进入报表中心，生成 PDF	生成成功，文件可下载，内容与交易数据一致
10	进入消费人格页面	人格画像卡片、四维雷达图、财务健康评分正常渲染
11	完成人格问卷自评	显示主观认知与交易数据画像的偏差对比
12	创建家庭组织并添加成员	组织创建成功，成员列表中显示被添加用户
13	查看家庭组织 Dashboard	显示家庭聚合收入、支出、交易数与家庭人格雷达图
14	查看系统设置页面	管理员可见 retry queue 状态与操作按钮，普通用户仅可见配置展示

每个验收步骤均在桌面视口（1440x900）下完成验证，确保页面无中文乱码、布局正常、交互响应正确。窄屏视口（390x844）下验证导航栏与内容区无严重重叠，表格与图表可正常展示。

7.4 发布状态

系统已完成部署并进入可用状态。部署架构采用 Docker Compose + Nginx 方案，各组件配置如下：

- **前端静态资源**：Vue 3 应用通过 Vite 构建为静态文件，由 Nginx 直接托管提供服务。
- **Nginx 反向代理**：监听 80 端口，前端路由由 Nginx 处理，/api 路径反向代理至后端 FastAPI 服务（127.0.0.1:8000）。
- **FastAPI 后端**：运行在内部 8000 端口，提供 RESTful API，由 Uvicorn 启动。
- **PostgreSQL**：持久化存储用户、交易、分类、组织、报表等业务数据。
- **Redis**：缓存与异步任务消息队列。低配部署时可通过 `TASK_ALWAYS_EAGER=true` 减少 Redis 依赖。

- **重试队列 Worker**: 通过 FastAPI lifespan 事件启动后台线程，串行处理标记为 `retry_queue` 状态的交易分类请求。

管理员账号通过后端 CLI 工具创建，不提供公开注册入口。生产环境配置包括环境变量注入（数据库连接、模型 API 密钥、Redis 地址等），通过 Docker Compose 的 `env_file` 或裸机部署的环境变量文件管理。

系统当前版本已交付的完整功能包括：用户认证与数据隔离、微信/支付宝账单导入与解析、大模型语义分类与人工校正、Dashboard 财务分析、消费人格画像与财务健康评分、家庭组织管理与聚合分析、PDF 报表生成与下载、多供应商分类切换与重试队列容错。系统具备生产级部署能力，后端测试覆盖率达到 75%，前端构建通过类型检查与生产构建验证。

第八章 运行与维护

本章阐述系统的运行环境配置、性能管理策略、可靠性保障机制以及日常运维检查要点。

8.1 运行环境

系统的运行时组件栈自上而下由以下部分构成：

浏览器端 用户通过现代浏览器访问单页应用，前端以静态资源形式部署，降低运行环境复杂度。

网关层 生产环境由 Nginx 统一提供静态文件服务和反向代理，并负责 HTTPS、压缩和缓存等通用能力。

后端服务 后端应用以内网服务形式运行，统一提供认证、导入、分类、分析、组织和报表等业务能力，并由进程管理工具保障持续运行。

数据与任务组件 生产环境使用 PostgreSQL 保存业务数据，Redis 支撑异步任务与缓存；轻量部署可采用 SQLite 和同步任务模式降低部署成本。

模型服务 分类模型既可接入外部大模型服务，也可切换至本地模型或规则兜底。模型请求由重试机制统一调度，避免外部服务波动直接影响用户操作。

文件存储方面，用户上传账单和生成报表分目录保存，并按用户维度隔离。运维上应定期清理过期上传文件，控制磁盘占用，同时保留可追溯的报表输出记录。

8.2 性能管理

下表汇总了各核心组件的已知性能瓶颈、当前缓解措施及未来优化方向：

表 8.1 性能管理汇总

组件	瓶颈描述	当前缓解措施	未来优化方向
账单导入解析	读取与转换大文件时占用 I/O 和内存	按文件分批处理，限制扫描范围，避免长事务	大文件异步处理，并行解析与批量写入
模型语义分类	外部模型延迟和可用性波动明显	统一重试池管理外部调用；缓存重复分类结果	引入语义相似缓存；支持批量分类减少网络往返
Dashboard 聚合	趋势与分类统计需扫描交易记录	使用索引和日期范围减少扫描量	引入物化视图或预聚合表，定时刷新统计
消费人格计算	Shannon 指数与余弦相似度涉及全量交易扫描	当前规模下（单用户数千笔）耗时百毫秒级，可接受	引入增量计算，新交易更新统计量而非全量扫描
PDF 报表生成	多页报表渲染消耗 CPU 时间	典型报表秒级生成；异步任务避免阻塞用户请求	超大型报表分页生成后合并
前端加载性能	组件库和图表库体积较大	生产构建启用按需优化与压缩	路由级代码分割，按页面懒加载图表能力

8.3 可靠性保障

系统通过以下五项机制保障数据处理的可靠性：

1. **重复交易控制**。每笔交易入库前都会生成稳定指纹，并按用户维度进行唯一性校验。同一文件重复导入时，系统自动跳过重复记录，避免统计结果膨胀。
2. **统一重试池**。外部模型调用与用户请求解耦，失败请求进入后台重试流程。重试达到上限后，交易被标记为需要管理员关注，避免失败任务持续占用处理资源。
3. **导入批次状态机**。导入任务按等待、处理中、完成、部分失败和失败等状态流转，页面通过进度条和状态标签向用户解释当前处理结果。
4. **报表任务状态机**。报表生成作为异步任务执行，用户可在生成完成后下载，避免大报表生成阻塞页面操作。
5. **权限校验**。系统在交易查询、分类修正、批次删除、报表下载和组织管理等操作前校验身份与资源归属，防止跨用户或跨组织访问。

8.4 运维检查

日常运维中建议按照以下检查清单定期审查系统状态：

表 8.2 运维检查清单

检查项	检查方式	预期结果与说明
日志监控	定期查看访问日志与错误日志	关注 5xx 错误频率、超时日志和异常堆栈，定位异常用户流程
健康检查	使用健康检查端点或监控系统探活	服务应持续返回正常状态，异常时及时告警
数据库备份	配置自动备份并定期演练恢复	生产环境建议每日备份，保留最近 7 天并验证可恢复性
密钥轮换	按计划更新认证密钥和模型访问凭证	轮换后旧凭证失效，建议低峰期执行并保留回滚方案
依赖安全审计	定期检查前后端依赖漏洞报告	关注高危漏洞，及时更新受影响的依赖版本
上传文件控制	检查文件大小、类型限制和存储目录	确保符合安全策略，定期清理过期上传文件
权限回归测试	修改认证或组织逻辑后运行相关测试	确保用户隔离和家庭角色权限未退化
重试队列监控	管理员查看重试状态与失败数量	若失败数量持续增长，应检查外部模型服务可用性
磁盘空间检查	定期检查上传文件和报表目录容量	不应超过预期；建议定时清理过期上传文件

此外，建议在生产服务器上配置以下监控：

- **进程存活监控**：确保前端网关、后端服务和后台任务异常退出后能够自动重启。
- **内存与 CPU 监控**：关注导入、报表生成和模型调用期间的资源尖峰，及时发现内存泄漏或计算瓶颈。
- **磁盘 I/O 监控**：关注上传文件、报表输出和数据库写入对磁盘的影响，必要时切换至更适合并发写入的数据库方案。
- **外部模型可用性告警**：若重试失败数量持续超过阈值，应触发告警通知运维人员检查模型服务可用性与配额状态。

第九章 总结与改进计划

9.1 项目完成情况

Boring Financial 智能账单分类系统在四个月的开发周期内，完成了从需求分析、系统设计、编码实现到测试部署的完整流程交付，兑现了项目启动阶段设定的核心目标。

在功能层面，系统实现了完整的账单分析闭环：用户可上传微信或支付宝账单文件，系统自动完成格式识别、交易解析、语义分类和低置信度交易提示；用户可在校正工作台复核分类结果，并在看板、消费人格页面和 PDF 报表中看到一致的分析口径。家庭组织模块在不改变个人账本归属的前提下，支持成员管理和家庭维度聚合分析，使系统从个人记账扩展到家庭协作场景。

在工程质量层面，系统建立了覆盖安全认证、账单解析、分类、分析、组织和报表等核心能力的自动化测试体系。后端 59 个测试用例全部通过，整体覆盖率达到 75%；前端 26 个测试用例、类型检查和生产构建均通过。系统同时提供本地运行、容器化部署和裸机部署方案，具备课程项目所需的可运行性、可测试性和可交付性。

在课程过程层面，项目完整经历了可行性分析、需求工程、界面原型、系统设计、程序开发、测试验收、发布部署和运行维护等阶段。团队在迭代过程中持续调整需求边界和技术方案，最终形成了功能闭环清晰、质量证据完整、后续改进方向明确的项目成果。

9.2 当前局限与后续改进

系统在以下方面仍存在改进空间：安全层面，上传校验、速率限制、HTTPS 强制和安全响应头部仍可继续加固；测试层面，前端已具备单元测试基线，但尚缺端到端集成测试覆盖完整用户流程；性能层面，大数据量下的看板聚合和前端资源体积仍有优化空间；运维层面，自动备份恢复和监控告警机制还需要进一步完善。

后续改进将按优先级分步推进：短期聚焦安全加固与前端端到端测试；中期引入看板缓存层与页面级资源优化；长期完善监控告警、自动备份与性能优化。系统当前版本已形成功能完整、架构清晰、可部署运行的个人与家庭账单分类分析产品。